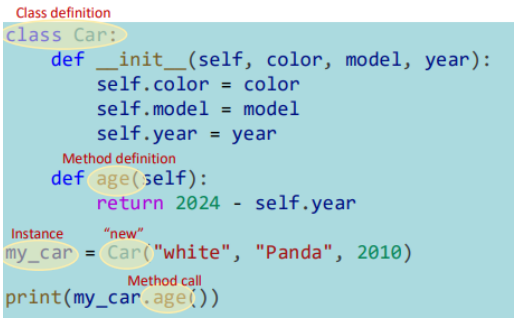
TdP 01 - object oriented programming in python

La classe rappresenta uno scheletro usato per costruire delle entità all'interno del codice (oggetti), ovvero entità astratte di entità fisiche realistiche che contengono una serie di proprietà e metodi propri dell'oggetto considerato, che in python risultano essere pubbliche, inoltre in python le variabili non sono tipizzate, in quanto in python qualsiasi cosa è un oggetto e l'oggetto è il contenuto delle variabili.
I costruttori sono delle funzioni utilizzate per creare un'istanza dello scheletro definito, in python fanno parte dei metodi speciali o dunder methods (metodi con doppio underscore), ovvero metodi che permettono di definire come l'oggetto considerato si comporta nel mondo, per cui vengono definiti dei protocolli per gli oggetti che comprendono una serie di caratteristiche minime che gli oggetti devono avere per essere integrati nel resto del codice.

La classe viene introdotta con la keyword *class* seguita dal nome della classe avente la prima lettera maiuscola per distinguerla dalle variabiliti, la proprietà della classe sono definite direttamente nel costruttore, ovvero una funzione definita dalla notazione def *__init__* che prende come argomenti tutti i parametri che servono a creare quell'oggetto, crea i parametri e assegna le variabili, il parametro *self* rappresenta un riferimento all'oggetto, in seguito sono presenti i metodi che agiscono sull'istanza e sono definiti come *def nome_metodo(self)*.
Self rappresenta una keyword che deve essere passata come primo argomento ad ogni metodo della classe e serve come reference a quale istanza dell'oggetto ci stiamo riferendo, per cui quando viene chiamato un metodo viene istanziato il self all'istanza dell'oggetto su cui la stiamo chiamando e quindi si ha accesso a tutti i metodi e proprietà di quella specifica istanza.
nota: attributi di classe restano uguali per tutte le istanze della classe, inserite all'interno della definizione della classe come *nome=valore*, per cui la modifica su variabili di classe si riversa su tutte le istanze della classe
nota: è importante mettere come attributo self in quanto rappresenta l'istanza dell'oggetto da cui il metodo va a leggere la proprietà



Esistono dei metodi per rendere delle variabili parzialmente private, mettendo uno (_) o due underscore (__) davanti alla variabile, quindi per accedere alle proprietà bisogna usare i metodi *@property* (equivalente getter) e *@name_proprietà.setter* (equivalente setter) introdotto dopo il metodo getter, che devono essere associati ad una funzione denominata come la variabile di cui si vuole creare il getter e il setter (come il nome senza _) *def nome_proprietà(self, …):*, utili per inserire dei controlli sulle variabili, quindi bisogna inizializzare una variabile *self._nome = None* su cui viene chiamato il getter e setter e una variabile *self.nome = nome* che poi verrà utilizzata per fare operazioni.
All'interno di una classe si possono definire metodi normali, ovvero metodi di quel istanza utilizzando def e self, metodi statici che non sono associati a nessuna istanza, per cui non bisogna passarli il self e vengono introdotti utilizzando il decoratore *@staticmethod*, e metodi di classe che si riferiscono all'intera classe e non all'istanza che si introducono utilizzando il decoratore *@classmethod* e il parametro cls come primo argomento, per cui il metodo standard viene chiamato come *nome_oggetto.metodo*, il metodo della classe come *nome_classe.metodo*.

I metodi speciali o dunder methods servono per personalizzare il comportamento degli oggetti e vengono introdotti da un doppio underscore (prima e dopo il nome del metodo), tra cui *__str__(self)* che restituisce una rappresentazione stampabile a stringa dell'oggetto (equivalente toString), *__repr__(self)* che restituisce una rappresentazione stampabile orientata al programmatore, per cui deve essere più rappresentativa dell'oggetto, ovvero una rappresentazione a stringa di come viene chiamato il costruttore dell'oggetto per ottenere l'oggetto, *__eq__(self, other)* che viene chiamato da == e

definisce se l'istanza di una classe è uguale ad un'altra istanza di una classe, *__lt__(self, other)* che rappresenta l'operatore <.
nota: per lista tutti metodi https://docs.python.org/3/reference/datamodel.html#special-method-names
nota: quando si utilizza print python usa di default __str__ se definito, mentre __repr__ viene utilizzato per descrivere la classe quando in debug mode

```
class Car:
    # ...

    def __str__(self):
        return f"{self.make}, {self.model}, {self.color}: ({self.year})"

    def __repr__(self):
        return (
            f"{type(self).__name__}"
            f'(make="{self.make}", '
            f'model="{self.model}", '
            f"year={self.year}, "
            f'color="{self.color}")'
        )
```

```
object.__add__(self, other)
object.__sub__(self, other)
object.__mul__(self, other)
object.__matmul__(self, other)
object.__truediv__(self, other)
object.__floordiv__(self, other)
object.__mod__(self, other)
object.__divmod__(self, other)
object.__pow__(self, other[, modulo])
object.__lshift__(self, other)
object.__rshift__(self, other)
object.__and__(self, other)
object.__xor__(self, other)
object.__or__(self, other)

object.__neg__(self)
object.__pos__(self)
object.__abs__(self)
object.__invert__(self)

object.__complex__(self)
object.__int__(self)
object.__float__(self)
```

```
object.__radd__(self, other)
object.__rsub__(self, other)
object.__rmul__(self, other)
object.__rmatmul__(self, other)
object.__rtruediv__(self, other)
object.__rfloordiv__(self, other)
object.__rmod__(self, other)
object.__rdivmod__(self, other)
object.__rpow__(self, other[, modulo])
object.__rlshift__(self, other)
object.__rrshift__(self, other)
object.__rand__(self, other)
object.__rxor__(self, other)
object.__ror__(self, other)

object.__round__(self[, ndigits])
object.__trunc__(self)
object.__floor__(self)
object.__ceil__(self)

object.__index__(self)
```

Per far ereditare una classe da un altra classe viene passata come argomento nella definizione della classe, per cui la classe eredita tutti gli attributi e metodi della classe padre *class NomeClasse(ClassePadre)*, in modo da specializzare il loro contenuto, quindi bisogna chiamare il costruttore della classe padre prima di inserire altri parametri all'interno del costruttore della classe figlia come *ClassePadre.__init__()* o *super().__init__()* contenenti al loro interno tutte le proprietà della classe Padre.
nota: in python è possibile ereditare da due classi chiamando entrambi i costruttori

Il polimorfismo è una proprietà della programmazione ad oggetti per la quale si vuole chiamare lo stesso metodo con diversi tipi di dati, possibile attraverso la strategia detta duck typing (If it walks like a duck and it quacks like a duck, then it must be a duck), secondo cui il tipo o la classe di un oggetto è meno importante dei metodi che la classe offre.
In una funzione polimorfica, ovvero una funzione che può implementare diversi classe, è possibile sapere il tipo effettivo della classe attraverso il metodo *isinstance(object, classinfo)*.
Per far sapere i metodi che devono essere implementati bisogna utilizzare un protocollo, ovvero uno strumento che ci viene fornito dalla libreria di typing che ci permette di specificare quali siano i metodi che ci aspettiamo che le classi abbiano per funzionare nel codice, ovvero è un metodo per rendere esplicito a livello di tipo quello che già si sta facendo a livello di comportamento nella definizione del codice.
Protocols è una classe astratta che presenta il solo obiettivo di definire la forma attesa per definire le altre classi che poi vengono utilizzate all'interno del codice, per cui bisogna importa protocol da typing (*from typing import Protocol*) e andare a definire una funzione figlia di protocol avente al suo interno solo la definizione metodi che le classi devono avere con un placeholder (...), in quanto si va a delegare l'implementazione del metodo a tutte le classi che devono utilizzare tale protocollo.

Una classe ben definita deve definire una serie di metodi di tipo speciale, tra cui:

- __init__() che contiene una serie di propeità self.name inizalizzate nel construttore
- __repr__ e __str__ per la rappresentazione a stringa dell'oggetto
- __eq__ per premettere le comparazioni con == e !=
- __lt__ se è necessario fare comparazioni del tipo < > <= >=
- __hash__ per utilizzare i dizionari
- metodi setter e getter sugli attributi se necessario
- tutti i metodi specifici della classe che specificano come la classe si comporta

Esistono degli strumenti per generare automaticamente codice, in caso di classe che contengano puro dato, attraverso il metodo *@dataclass* che deve essere importato (*from dataclasses import dataclass*)e si può utilizzare per scrivere una classe che conterrà solo dati, per cui a seguito del decoratore vengono listati tutti i parametri con i tipi.



nota: se un metodo deve prelevare qualcosa da una classe allora è bene chiamarlo get_qualcosa, se invece aggiunge qualcosa alla classe allora è bene chiamarlo add_qualcosa, ovvero bisona cercare di usare nomi esplicativi

```
class Prodotto:
   aliquota_iva = 0.22
   #variabile di classe, ovvero è la stessa per tutte le istanze che verrano create
   def __init__(self, name: str, price: float, quantity: int, supplier = None):
       #è possibile suggerire il tipo della variabile
       #è possibili avere la notazione = per fornire un valore di default nel caso l'utente  non
imposti un valore
       self.name = name
       #per rendere una variabile parzialmente privata si usa _ o __
       self._price = None
       self.price = price
       self.quantity = quantity
       self.supplier = supplier
   #metodi normali associati all'istanza, per cui deve associare il self!!!
   def valore_netto(self):
       return self.price*self.quantity
   def valore_lordo(self):
       netto = self.valore_netto()
       lordo = netto*(1+self.aliquota_iva)
       return lordo
   #è possibile usare dei metodi di classe che utilizzano cls al posto di self
   #mettendo prima della definizione il decoratore introdotto dalla ciocciola:
   @classmethod
   def costruttore_con_quantita_uno(cls, name: str, price: float, supplier: str):
       cls(name, price,1, supplier)
   #i metodi statici vengono instrodotti con il decoratores staticmethods e vengono definiti come
le altre funzioni
   @staticmethod
   def applica_sconto(price, percentuale):
       return price*(1-percentuale)
   #per accedere alla variabile parzialmente privata si utilizza il metodo:
   @property
   def price(self): #equivalente getter
       return self._price
   @price.setter
   def price(self, value): #equivalente setter
       if value<0:
           raise ValueError("Attenzione il prezzo non può essere negativo")
       self._price = value

   def prezzo_finale(self):
       return self.price*(1+self.aliquota_iva)
   def __str__(self): #equivalente a tostring
       return f"{self.name} - disponibilità {self.quantity} pezzi a {self.price} $"
   def __reper__(self):
       return f"Prodotto(nome={self.name}, price={self.price}, quantity={self.quantity},
supplier={self.supplier})"
   def __eq__(self, other: object):
       if not isinstance(other, Prodotto):
           return NotImplemented
       return {self.name==other.name and self.price==other.price
               and self.quantity==other.quantity and self.supplier==other.supplier}
   def __lt__(self, other: "Prodotto") -> bool:
       #utilizzo la -> per anticipare il tipo del risultato
       return self.price < other.price

class ProdottoScontato(Prodotto):
   def __init__(self, name: str, price: float, quantity: int, supplier: str, sconto_percento:
float):
       super().__init__(name, price, quantity, supplier)
       self.sconto_percento = sconto_percento
def prezzo_finale(self) ->float:
   return self.valore_lordo()*(1-self.sconto_percento/100)

class Servizio(Prodotto):
   def __init__(self, name: str, tariffa_oraria: float, ore: int):
       super().__init__(name=name, price=tariffa_oraria, quantity=1, supplier=None)
       self.ore = ore
   def prezzo_finale(self) ->float:
       return self.price*self.ore

p1 = Prodotto(name="Laptop", price=1200.0, quantity=12, supplier="ABC")
print(f"Nome prodotto: {p1.name} - prezzo: {p1.price}")
```

```python
print(f"Il totale lordo di p1 è {p1.valore_lordo()}") #uso metodo di istanza
#per usare medoti statici
print(f"Prezzo scontato di p1 {Prodotto.applica_sconto(p1.price, 0.15)}")
p2 = Prodotto(name="Mouse", price=10, quantity=25, supplier="CDE")
print(f"Nome prodotto: {p2.name} - prezzo: {p2.price}")
#per chiamare un metodo di classe
p3 = Prodotto.costruttore_con_quantita_uno("Auricolari", 200, "ABC")
print(f"Valore lordo di p1: {p1.valore_lordo()}")
Prodotto.aliquota_iva = 0.24
print(f"Valore lordo di p1: {p1.valore_lordo()}")

print(p1) #py utilizza per default __str__
p_a=Prodotto(name="Laptop", price=1200.0, quantity=12, supplier="ABC")
p_b=Prodotto(name="Mouse", price=10, quantity=14, supplier="CDE")
print(f"p1 == p_a? {p1==p_a}") #chiama il metodo __eq__ appena implementato
print(f"p1 == p_b? {p1==p_b}")
myList=[p_a, p_b, p1]
myList.sort() #ordine crescente
myList.sort(reverse=True) #ordine descrescente
print("lista di prodotti ordinata")
for p in myList:
   print(f"- {p}")

pScontato1 = ProdottoScontato("auricolari", 320, 1, "ABC", 10)
servizio1 = Servizio("consulenza", 100, 3)
myList.append(pScontato1)
myList.append(servizio1)
myList.sort(reverse=True)
for element in myList:
   print(element.name, "->", element.prezzo_finale())

from typing import Protocol
#definisco una classe figlia di protocol avente solo i metodi che le classi devono avere
class HaPrezzoFinale(Protocol):
   def prezzo_finale(self)->float:
       ... #questo è un placeholder

def calcola_totale(elementi: list[HaPrezzoFinale])->float:
   return sum(e.prezzo_finale() for e in elementi)
print(f"il prezzo totale è: {calcola_totale(myList)}")

print("------------------------------------------------------------")

from dataclasses import dataclass
@dataclass
class ProdottoRecord:
   name: str
   prezzo_unitario: float
@dataclass
class ClienteRecord:
   name: str
   email: str
   categoria: str
@dataclass
class RigaOrdine:
   prodotto: ProdottoRecord
   quantita: int
   def totale_riga(self):
       return self.prodotto.prezzo_unitario*self.quantita
@dataclass
class Ordine:
   righe: list[RigaOrdine]
   cliente: ClienteRecord
   def totale_netto(self):
       return sum(r.totale_riga() for r in self.righe)
   def totale_lordo(self, aliquota_iva):
       return self.totale_netto()*(1+aliquota_iva)
   def numero_righe(self):
       return len(self.righe)
@dataclass
class OrdineConSconto(Ordine): #è possibile ereditare da altre classi
   sconto_percentuale: float
   def totale_scontato(self):
       return self.totale_lordo()*(1-self.sconto_percentuale)
   def totale_netto(self): #è possibile modificare metodi della classe padre
       netto_base = super().totale_netto()
       return netto_base*(1-self.sconto_percentuale/100)
```

```python
c1 = ClienteRecord("Mario Bianchi", "mario.bianchi@gmail.com", "Gold")
p1 = ProdottoRecord("laptop", 1200.0)
p2 = ProdottoRecord("mouse", 20)
ordine = Ordine([RigaOrdine(p1,2), RigaOrdine(p2, 10)], c1)
ordine_scontato = OrdineConSconto([RigaOrdine(p1,2), RigaOrdine(p2, 10)], c1, 10)
print(ordine)
print("numero di righe nell'ordine", ordine.numero_righe())
print("totale netto:", ordine.totale_netto())
print("totale lordo (IVA 22%):", ordine.totale_lordo(0.22))
print(ordine_scontato)
print("totale netto scontato: ", ordine_scontato.totale_netto())
print("totale lordo scontato: ", ordine_scontato.totale_lordo(0.22))

print("------------------------------------------------------------")

#scrivere classe cliente che abbia i campi nome, email, categoria (gold, silver, bronze)
#la classe deve avere un metodo nominato descrizione che deve restituire una stringa formattata
#ad esempio: "Cliente Fulvio Bialnchi (Gold) - fulvio@gmail.com"
#si modifichi la classe cliente in maniera tale che la proprietà categoria sia "protetta"
#e accettis solo (Gold, Silver, Bronze)

class Cliente:
    def __init__(self, name: str, email: str, categoria: str):
        self.name = name
        self.email = email
        self._categoria = None
        self.categoria = categoria
    def descrizione(self): #toString
        return f"Cliente {self.name} ({self.categoria}) - {self.email}"
    @property
    def categoria(self):
        return self._categoria
    @categoria.setter
    def categoria(self, value):
        categorie_valide = {"Gold", "Silver", "Bronze"}
        if value not in categorie_valide:
            raise ValueError("Categoria non valida. Scegliere fra Gold, Silver, Bronze")
        self._categoria = value

c1 = Cliente(name="Mario Bianchi", email="mario.bianchi@gmail.com", categoria="Gold")
#c2 introdotto per inalzare errore e quindi commentato
#c2 = Cliente(name="Carlo Masone", email="carlo.masone@gmail.com", categoria="Platino")
print(c1.descrizione())
```

TdP 1.b - GitHub

Le repository è un database in cui vengono salvate tutte le variazioni fatte sul lavoro, per cui conterrà tutto il file, la struttura delle directories (le cartelle e il loro contenuto) e la storia delle cartelle e dei file, per cui è possibile andare a rivedere il file in un tempo arbitrario.
L'ultima versione, su cui applichiamo le modifiche, viene detta working copy, per cui avremmo due entità, ovvero la working copy ce rappresenta la struttura di codice su cui si sta lavorando e il repository che comunicheranno tra di loro attraverso l'operazione di **commit** per cui la working copy attuale viene salvata nella repository e quindi va a modificare la storia del codice, per cui bisogna sempre associare un messaggio obbligatorio (commento) in cui per futura reference viene descritto cosa è stato fatto in modo che il repository sappia che cosa sia stato fatto in ogni commit.
La repository viene messe online in un server che offre un servizio di git, per integrare le modifiche fatte sulla working copy anche da più utenti, in caso di conflitti, ovvero più utenti che hanno modificato la stessa riga di codice, bisogna decidere manualmente che versione tenere.
È possibile avere un sistema di it distribuito, per cui ogni utente presenta una versione della working copy e della repository, per cui a un certo punto le varie repository dovranno essere allineate tramite l'operazione di **push**, per cui le modifiche recepite dal database locale vengano recepite anche dal server remoto ce osta il repository, inoltre esiste l'operazione inversa di **pull** che permette di scaricare le modifiche fatte al repository remoto sul repository locale.

Operazioni per scaricare progetto esistente:

1) **fork** del progetto desiderato in github (crea una copia nel proprio repository)
   sync fork per aggiornare le modifiche che sono state fatte alla repository (pull)
2) **<>code** copiare l'URL della repository dal nostro github

3) per importare la repository sul computer locale (creare una copia locale) → andare su PyCharm → **file** → **project from version control** → come URL quella della repository copiata da github, come Directory la cartella su cui si vuole salvare la repository→ **clone**
   tasto **main** (in alto vicino nome lab) con tutti i comandi di github
4) per creare nuovo progetto nel file → tasto destro nome file → new → python → add
   per salvare progetto o uso tasto main oppure → tasto destro sul nome cartella → **git** → **commit directory** → aggiungi messaggio → **commit**
   in seguito effettuare l'operazione di push → tasto destro sulla cartella → **git** → **push**

Operazioni per caricare progetto sulla nostra macchina online:

1) creare una repository vuota → andare sul proprio account personale → **repository** → **new** → inserire parametri
2) per caricare progetto copia URL relativo al proprio github e andare sul progetto da salvare su pycharm → **VSC** → **create git repository** nella cartella attuale → **open**
3) per aggiungere la repository a github **git** → **manage remotes** → **+** inserire link della repository → **ok**
4) operazione di commit e push come precedentemente descritto oppure usare direttamente commit and push

TdP 02 - py modules

Può essere utile dividere il codice in moduli per rendere il codice più modulare/leggibile e rendere le nostre classi componenti di una libreria di classi che si possono riutilizzare in altri progetti, per cui i moduli sono delle collezioni di classi, funzioni e variabili, per creare dei moduli bisogna creare dei file.py all'interno della directory contenente le vari classi utilizzando il comando import per importare i nomi presenti in un certo modulo
nota: i nomi che definiamo in un modulo vengono creati in un namespace diverso, per cui nel programma si può decidere cosa importare per evitare di mischiare nomi con funzionalità diverse
nota: import va a leggere tutto il codice all'interno del modulo ed esegue tutto il codice eseguibile, per cui un modulo ben scritto non contiene codice eseguibile fuorni a funzioni o classi, quindi il codice eseguibile creato per testare il modulo deve essere salvato all'interno di una variabile *def _test_modulo():*, in modo che python esegui il codice solo se il codice non è importato all'interno di main.

```
if __name__=="__main__":
        _test_modulo()
```

Modi per importare i moduli:

- *from nome_modulo import name(s)*
- *from nome_modulo import name as alt.name* per rinominare il nome che si sta importando
- *import nome_modulo* che importa tutti i prodotti dentro main
- *import nome_modulo as alt_name*

nota: nei casi di import diretti per accedere a tutto quello che è stato importato bisogna usare il suffisso *nome_modulo*
La funzione *dir()* restituisce una lista di tutti i nomi contenuti nel modulo corrente, mentre *dir(nome_modulo)* restituisce una lista di tutti i nomi contenuti nel modulo importato

I packages sono delle collezioni di moduli che possono essere importati come i moduli e vengono creati seguendo i passaggi new → python package che al suo interno avrà un file __init__ vuoto creato automaticamente per specificare che si tratta di un package, inoltre è possibile avere dei sub package per ordinare a lora volta i package.
Modi per importare i package:

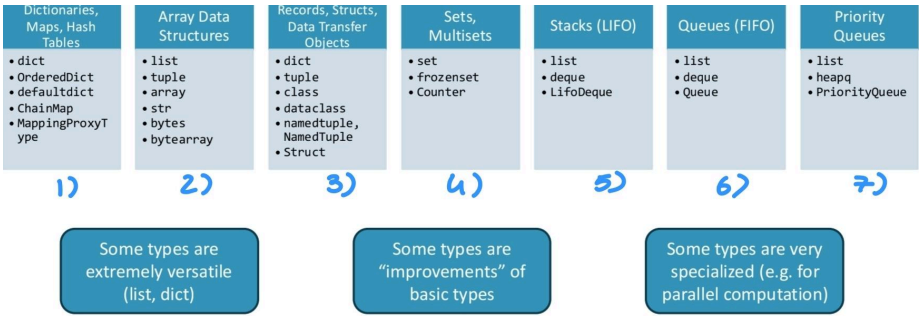
- *import pkg.mod*
- *from pgk.mod import name*
- *from pgk.mod import name as alt_name* tipicamente usato
- *from pkg import mod*
- *from pkg import mod as alt_mod_name*

Per lavorare con package esterni (librerie) bisogna tipicamente installarli facendoci aiutare da pycharm (*import networkx as nx*) o utilizzando il comando *pip install project_name* dal terminale (terzo pulsante dal basso a sinistra), inoltre è possibile utilizzare python package (quinto dal basso a sinistra) in cui si può cercare la libreria da installare.
Le librerie presentano versioni diverse che possono entrare in conflitto tra di loro se installate sullo stesso progetto, per risolvere questa problematica vengono usati i **virtual environment**, per cui per ogni progetto sarà presente un virtual environment diverso, ovvero una sandbox in cui vengono scaricate tutte e sole le librerie che servono al progetto per evitare conflitti.
Per essere sicuri di usare le versioni delle librerie che vogliamo utilizzare un file di testo detto *requirement.txt* all'interno del quale sono listate tutte le librerie e le loro versioni da utilizzare, per cui pycharm riesce a capire quali librerie non sono installate e ti propone di installarle tutte.
nota: all'esame ricordarsi di andare sul file requirements.txt ed installare le librerie

Passaggi per creare un virtual environment:
cliccare in basso a destra su python 3.14 → **add new interpreter** → **add local interpreter** → environment: **generate new** → type: **virtual environment** → crea la cartella .venv e sotto sarà presente python 3.14 (lezione 02)

TdP 03 - collections

Esistono diverse strutture dati in python che permettono di creare dei contenitori di oggetti, tra cui le liste che rappresenta una successione più o meno ordinata di oggetti, le tuple, i set e i dizionari, che sono specializzati in funzione dello scopo del codice.
I dizionari instazioati come dict sono una struttura che ci permette di indicizzare oggetti come target di una chiave che viene associata all'oggetto, per cui è possibile andare a riprendere un oggetto tramite una chiave, per cui ha senso usarla nel caso in cui si usa la collection per salvare dei dati e poi dobbiamo andare a prendere un particolare elemento in posizione casuale e non si vuole aspettare il tempo di scorrere tutta una lista.
In certi casi ha senso mantenere un ordinamento, tutte le strutture che prevedono un ordinamento vengono dette array e contengono le liste, tuple, stringhe.
Inoltre esistono i record, ovvero un'entità che contiene dei campi a cui si associano dei valori, tra cui classi e dizionari, esistono anche delle funzionalità più avanzate che servono per modellizzare le code LIFO (liste, deque, lifodeque), code FIFO (list, deque, queue), dove il deque è una lista con due entrypoint, code prioritarie (list, heapq, priorityQueue).



Schema sinottico delle principali operazioni sui contenitori

Operation	str	list	tuple	set	dict
Create	`"abc" 'abc'`	`[a, b, c]`	`(a, b, c)`	`{a, b, c}`	`{a:x, b:y, c:z}`
Create empty	`"" ''`	`[] list()`	`() tuple()`	`set()`	`{} dict()`
Access i-th item	`s[i]`	`l[i]`	`u[i]`		`d[k]` `d.get(k,default)`
Modify i-th item		`l[i]=x`			`d[k]=x`
Add one item (modify value)		`l.append(x)`		`t.add(x)`	`d[k]=x`
Add one item at position (modify value)		`l.insert(i,x)`			
Add one item (return new value)	`s+'x'`	`l+[x]`	`u+(x,)`		
Join two containers (modify value)		`l.extend(l1)`		`t.update(t1)`	
Join two containers (return new value)	`s+s1`	`l+l1`	`u+u1`	`t.union(t1) t|t1`	
Does it contain a value?	`x in s`	`x in l`	`x in u`	`x in s`	`k in d` (search keys) `x in d.values()` (search values)
Where is a value? (returns index)	`s.find(x)` `s.index(x)`	`l.index(x)`	`u.index(x)`		
Delete an item, by index		`l.pop(i) l.pop()`			`d.pop(k)`
Delete an item, by value		`l.remove(x)`		`t.remove(x)` `t.discard(x)`	
Sort (modify value)		`l.sort()`			
Sort (return new **list**)	`sorted(s)`	`sorted(l)`	`sorted(u)`	`sorted(t)`	`sorted(d)` (keys) `sorted(d.items())`

L'ordinamento di una collection funziona sfruttando due dunder methods che devono essere inseriti nella definizione della classe, ovvero __eq__ che definisce se due oggetti sono uguali o no, __lt__ che restituisce un valore vero/falso se si può definire che un oggetto è minore di un'altro.
Alcune collection presentano già per loro costruzione delle implementazioni built-in di __eq__ e __lt__, tra cui stringh, liste e tuple che comparano i loro elementi in ordine da sinistra a destra, i dizionari supportano __eq__ ma non __lt__, i set supportano sia __eq__ ch __lt__, tuttavia in questo caso __lt__ indicherà un subset del set considerato (insieme 1 contenuto dell'insieme 2).
La @dataclass presenta già di default il metodo __eq__ che andrà a confrontare le singole proprietà della classe, per cui se tutti i campi sono uguali allora le due istante della dataclass sono uguali, mentre non definisce un metodo __lt__ che dovrà essere definito, oppure come argomento al decoratore si può inserire order=True (*@dataclass(order=True)*) che permette di delegare l'ordinamento all'ordinamento dei vari campi, inoltre il decoratore può assumere come argomento *eq=False* per cui non andrà a creare il metodo __eq__.
nota: uno o più campi possono essere omessi dai metodi di confronto e ordinamento, inizializzandoli con *field(compare=False)*

Il sorting può avvenire secondo metodi diversi, per cui se si vuole ordinare secondo una specifica proprietà della collezione si utilizza il metodo attrgetter del modulo operator come chiave, il metodo più generale per fare sort corrisponde all'utilizzo di una lambda function, ovvero secondo una funzione creata da noi, che premette di fare dei sorting più complicate.

- *key=operator.itemgetter('keyname')* – sort by dictionary['keyname']
- *key=operator.itemgetter(itemnumber)* – sort by list/tuple[itemnumber]
- *key=operator.attrgetter('attrname')* – sort by object.attrname
- *key = lambda obj: something(obj)* (ex. list.sort(key=lambda x: x.proprietà))

1. dictionaries
Un dizionario è una struttura dati che mappa due elementi, chiave e valore con sintassi { key: val }, per cui la chiave deve essere hashable, ovvero essere dotata di una funzione di hash, quindi una funzione di mapping in cui ad un elemento si associa un valore di tipo intero che deve essere univoco dell'argomento in ingresso.
Un oggetto è hashable se posso calcolare un valore di hash immutabile nel tempo, inoltre l'oggetto deve essere comparabile, per cui bisogna definire il metodo __eq__, per cui due istanze di una classe sono uguali se presentano hash uguale ($a == b \Rightarrow hash\ a == hash\ b$).
Per assicurarci che gli oggetti abbiano una funzione di hash si utilizza il metodo __*hash*__ che restituisce un intero in output che definisce in maniera univoca quell'istanza, si può delegare la funzione di hash alle variabili come nel confronto, per cui la funzione di hash dell'oggetto si ottiene come funzione di hash di una tupla contenente tutte le proprietà dell'oggetto che ci interessa.
La funzione di hash è una funzione che mappa ogni oggetto in un intero su 64bit.
Se si riesce a mappare ogni oggetto del programma, ovvero tutte le istanze delle classi, in hash diversi allora sarà possibile effettuare una ricerca molto efficiente, in quanto è possibile utilizzare quella funzione di hash come indirizzo di memoria, per cui i dizionari sono efficienti per inserire valori, cercarli e toglierli senza badare all'ordinamento.

`d[key] = value`	Set a new value for a key
`d[key]`	Retrieve value from the key. May raise `KeyError`
`d.clear()`	Clears a dictionary.
`d.get(key, default)`	Returns the value for a key if it exists in the dictionary. Otherwise, returns a default value
`d.items()`	Returns a list of key-value pairs in a dictionary.

`d.keys()`	Returns a list of keys in a dictionary.
`d.values()`	Returns a list of values in a dictionary.
`d.pop(key, default)`	Removes a key from a dictionary, if it is present, and returns its value. Otherwise, returns a default value
`d.popitem()`	Removes the last key-value pair from a dictionary.
`d.update(obj)`	Merges a dictionary with another dictionary

Altri dizionari, specificazione dello standard:

- ***d = collections.defaultdict(int)*** sono delle implementazioni dei dizionari standard dove se si prova a leggere una chiave che non esiste, automaticamente quella chiava e quel valore vengono creati e il valore viene assegnato

come valore di default; un defaultdict viene creato come oggetto del modulo collection, che dovrà essere importato, e bisogna passargli un argomento, ovvero un tipo che viene utilizzato per definire il valore di default

- *readonly_d = types.MappingProxyType(normal_d)* crea un dizionario partendo da un dizionario già creato (normal:d), ovvero mappa un dizionario in un nuovo dizionario che non si potrà modificare, per cui maschera tutti i metodo che vanno a modificare il dizionario generando un *TypeError: 'mappingproxy' object does not support item assignment*

2. array

Tipi principali:

- *list* rappresenta una sequenza ordinata secondo un certo criterio di oggetti, versatile e mutabile
- *tuple* è una versione immutabile di una lista, per cui una volta creata gli elementi non possono essere aggiunti o eliminati, ma possono essere modificati se si tratta di oggetti mutabili, inoltre la tupla è hashable se gli elementi della tupla sono dotati di funzione di hash, per cui l'hash della tupla sarà dato dalla combinazione delle funzione di hash degli oggetti
- *str* è una sequenza di caratteri immutabili di tipo ASI

Array specializzati:

- *array.array* è una tupla in cui all'inizio viene definito che cosa contiene, per cui bisogna specificare che tipi di elementi verranno passati e in seguito vengono passati gli elementi uno dopo l'altro
- *bytes* e *bytearray* array di oggetti di tipo byte di due versioni, mutabili o immutabili

3. records

I record sono delle collection che contengono istanze di elementi di tipo eterogeneo (ex. classe, dataclass, dict, tuple).

Record specializzati:

- *collections.namedtuple()* sono delle tuple a cui ad ogni elemento viene associato un nome, viene creato come classe del modulo collection, per cui è possibile accedere agli argomenti della tupla con il nome
- *typing.NamedTuple* presenta una sintassi simile alla dataclass e crea una classe che presenta dei parametri che si andranno a riempire

4. set

I *set* sono contenitori mutabili di oggetti hashable, avente particolarità che i duplicati non sono permessi, in quanto un set è un insieme di oggetti distinti avente sintassi *nome = {elementi}*, inoltre esiste una variante detta *frozenset* che rappresenta una versione immutabile del set, per cui una volta creata non si può modificare, e essendo immutabile può essere utilizzato come chiave di un dizionario.

I *counter* sono degli oggetti del modulo collections (*collections.Counter*), per cui crea un set di elementi distinti a cui viene associato quante volte quell'elemento viene passato come argomento, ovvero si comporta come un dizionario dove la chiave sono gli elementi del set e il valore quante volte sono stati passati, inoltre fornisce dei metodi utili per fare conti sulle frequenze, tra cui:

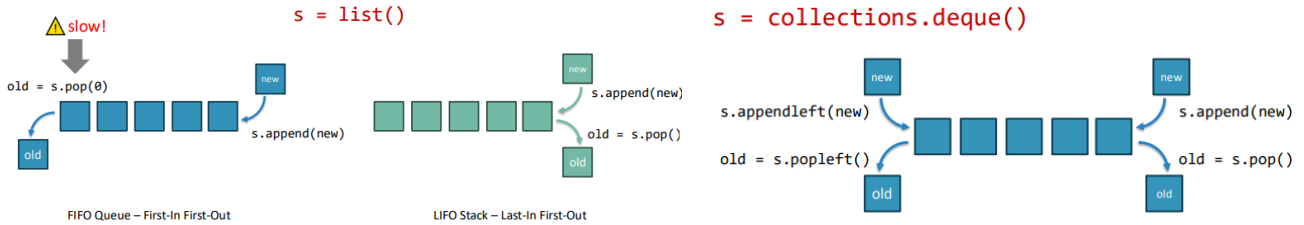
```
• c.most_common(n)               # the 'n' (default: all) most common items
• c.total()                      # total of all counts
• list(c)                        # list unique elements

• set(c)                         # convert to a set
• dict(c)                        # convert to a regular dictionary
• c.items()                      # convert to a list of (elem, cnt) pairs
• c.elements()                   # return a list [elem, …] with repetitions
• Counter(dict(list_of_pairs))   # convert from a list of (elem, cnt) pairs
• c.most_common()[:-n-1:-1]      # n least common elements
• +c                             # remove zero and negative counts
• c.clear()                      # reset all counts
```

nota: c.most_common() è possibile utilizzare questo metodo senza nessun argomento, per cui restituisce un ordinamento degli entry per frequenza, ovvero quante volte quel valore è stato fornito

5+6. stacks (LIFO) e queues (FIFO)

Le pile e code sono simili ad una lista avente modi diversi per accedere ai valori contenuti all'interno della collection, per cui nelle pile si assume che l'ultimo oggetto sia il primo ad essere estratto (LIFO), mentre nelle code si assume che il primo oggetto sia il primo ad essere estratto (FIFO).



È possibile rappresentarle con delle liste *s=list()*, per cui si usa *s.append(new)* per appendere un nuovo elemento e *old=s.pop()* per estrarre l'ultimo elemento (pile) o *old=s.pop(0)* per estrarre il primo elemento (code), tuttavia quest'ultima operazione risulta essere molto lenta (complessità di ordine n con n lunghezza della lista), per risolvere questa problematica si usano i deque (double-ended queue) *s = collections.deque()*, ovvero delle liste a cui si può aggiungere a sinistra e a destra senza differenza di complessità computazionale.

In tutti i casi in cui ha senso gestire le cose in ordine di arrivo, allora ha senso provare ad usare un deque

As As a FIFO Queue:

- append and popleft – Most popular choice
- appendleft and pop – Also possible, same efficiency

As a LIFO Stack:

- append and pop – Most popular choice, might use a list, instead
- appendleft and popleft – Also possible, same efficiency

`d = deque()`	New empty deque
`d = deque(iterable)`	Deque from list
`d = deque(maxlen=N)`	Hosts max N elements, discards older ones if more are added
`d.extend(iterable)`	Adds list of elements at end
`d.extendleft(iterable)`	Adds list of elements at beginning
`d.rotate(n)`	Rotate elements by n steps
`d[i]`	Access element (slower than lists)
`d.index(x), d.insert(i, x), d.remove(x), d.reverse()`	Same as lists

7. priority queues

Le code prioritarie sono delle collection simili ad una lista, per cui si aggiunge a destra con la differenza che quando si fa il pop della lista viene restituito l'oggetto con priorità più elevata, quindi quando si aggiungono degli oggetti bisogna passargli anche un altro valore che rappresenta la priorità dell'oggetto in modo da poter ordinare la lista a seconda della priorità, per cui è possibile creare una tupla (priority, value) e poi fare un sort della tupla per priorità, oppure definire un metodo __lt__ sull'oggetto che vada a pesare le priorità degli oggetti.

Il metodi heapq e queue.PriorityQueue permettono di gestire le priorità senza dover fare sorting con un algoritmo che costruisce una gerarchia ad albero tra gli oggetti della lista che associa il metodo pop all'oggetto avente priorità più elevata.

```
import copy
from collections import Counter, deque

# LIST
p1 = pr("laptop", 1200.0)
p2 = pr("mouse", 20.0)
p3 = pr("auricolari", 250.0)
carrello = [p1, p2, p3, pr("tabel", 700.0)]
# posso creare istanze dentro la lista, a cui posso accedere solo come elemento della
lista (non ha nome)
print("prodotti nel carrello:")
for i, p in enumerate(carrello): print(f"{i}) {p.name} - {p.prezzo_unitario}")
tot = sum(p.prezzo_unitario for p in carrello)
```

```python
# aggiungere ad una lista
# carrello.append(elem) #aggiunge in coda un elemento (come ultimo)
# carrello.insert(x, elem)#aggiunge un elemento nella posizione x
# carrello.extend([lista elem]) #aggiunge una lista di elementi
carrello.sort(key=lambda x: x.prezzo_unitario, reverse=True)

# rimuovere ad una lista
# carrello.pop() #rimuove l'ultimo elemento
# carrello.pop(2) #rimuove l'elemento nella posizione x
# carrello.remove(p1) #rimuove la prima occorrenza di p1
# carrello.clear() #svuota la lista

# sorting, tutti questi metodi modificano la lista carrello
# carrello.sort() # ordina seguendo ordinamento naturale,
                  # non funziona se gli oggetti contenuti non definiscono un metodo __lt__
# carrello.sort(reverse=True) # ordina in senso inverso
# carrello.sort(key=function)
# carello_ordinata = sorted(carrello) # se si vuole creare una lista nuova
# carrello.reverse() # restituisce la lista al contrario (inverte l'ordine)

# creare una copia
carrello_copy = carrello.copy()
# crea una shallow copy, ovvero contiene gli stessi oggetti del carrello
# s ono due liste diverse che contengono gli stessi oggetti
carrello_copy2 = copy.deepcopy(carrello) # deep copy, ovvero copio anche il contenuto
degli elementi
--------------------------------------------------------------------------------
#TUPLE
sede_principale = (45, 8) # latitudine e longitudine della sede di torino
# accedo agli elementi della tupla con []
print(f"sede principale ha lat {sede_principale[0]}, long {sede_principale[1]}")
# tupla di tuple
AliquoteIVA = (("standard", 0.22), ("ridotta", 0.10), ("alimentari", 0.04), ("esente",
0.0))
for i, valore in AliquoteIVA: print(f"{i}: {valore*100}%")
# fare return su una tupla
def calcola_statistiche_carrello(carrello):
       prezzi = [p.prezzo_unitario for p in carrello]
       return (sum(prezzi), sum(prezzi)/len(prezzi), max(prezzi), min(prezzi))
# tupla = calcola_statistiche_carrello(carrello)
tupla, media, max, min = calcola_statistiche_carrello(carrello) #fa l'unpacking degli
elementi della tupla
# tot, *altri_campi = calcola_statistiche_carrello(carrello) #solo il primo elemento
finisce in tot
--------------------------------------------------------------------------------
# SET
categorie = {"Gold", "Silver", "Bronze", "Gold"}
print(categorie, len(categorie))
categorie2 = {"Platine", "Elite", "Gold"}
categorie_all = categorie | categorie2 # unione e le assegna ad un nuovo nome
categorie_comuni = categorie & categorie2 # solo elementi comuni
categorie_escluzive = categorie - categorie2 # solo elementi presenti in uno dei due set
categorie_esclusive_simmetrico = categorie ^ categorie2 # differenza simmetrica

# CONTROLLA CHE GLI ELEMENTI ABBIANO UNA FUNZIONE DI HASH
s = {pr("laptop", 1200.0), pr("mouse", 20.0), pr("tablet", 700.0)}
s1 = set()
# metodi utili per set
# s.add(pr("auricolari", 250.0)) #aggiunge un elemento
# s.update([pr("aaa", 100.0), pr("bbb", 100.0)])#aggiunge più elementi
# s.remove(elem) #rimuove un elemento, raise keyerror se non esiste
# s.discard(elem) #rimuove un elemento senza sollevare errore se non esiste
# s.pop() #rimuove e restituisce un elemento
# s.clear() #rimuove il set
# operazioni insiemistiche
# s.union(s1) #s|s1 genera un set che unisce i due set di partenza
# s.intersection(s1) #s&s1 genera un set con elementi comuni
# s.difference(s1) #s-s1 genera set con elementi di s non contenuti in s1
```

```python
# s.symmetric_difference(s1) #s^s1 genera set con elementi di s non contenuti in s1 ed elementi di s1 non contenuti in s
# s1.issubset(s) #True se gli elementi di s1 sono contenuti in s
# s1.issuperset(s)  #True se gli elementi di s sono contenuti in s1
# s1.isdisjoint(s) #True se gli elementi di s e di s1 sono diversi
------------------------------------------------------------------------------
# DICTIONARY
catalogo = {"LAP001": pr("laptop", 1200.0), "LAP002": pr("laptop pro", 2300.0),
            "MAU002": pr("mouse", 20.0), "AUR001": pr("auricolari", 250.0)}
cod = "LAP001"
prod = catalogo[cod]
# print(f"cerco un altro oggetto: {catalogo["NonEsiste"]}") #solleva errore
p4 = catalogo.get("NonEsiste") #restituisce none
p5 = catalogo.get("NonEsiste2", pr("sconosciuto", 0.0)) #restituisce default

# ciclare su un dizionario
keys = list(catalogo.keys())
values = list(catalogo.values())
for k in keys: print(k)
for v in values: print(v)
for k, v in catalogo.items(): print(f"cod {k} è associata a {v}") # restituisce coppia nome-valore
# rimuovere dal dizionario
rimosso = catalogo.pop("LAP001")
# dict comprehension, ci permette di scrivere come deve essere costruito il dizionario direttamente come argomento delle {}
prezzi = {codice: prod.prezzo_unitario for codice, prod in catalogo.items()}
# metodi da ricordare per dict:
# d[key] = v # scrivo sul dizionario
# v = d[key] # leggere -- restituisce key error se non esiste
# v = d.get(key, default) # legge senza rischiare keyerror. Se non esiste rende il default
# d.pop(key) # restituisce un valore e lo cancella dal diz
# d.clear() # elimina tutto.
# d.keys() # mi restituisce tutte le chiavi definite nel diz
# d.values() # mi restituisce tutti i valori salvati nel diz
# d.items() # restituisce le coppie.
# key in d # condizione che verifica se key è presente nel diz
------------------------------------------------------------------------------
#COUNTER
lista_clienti = ["lista clienti..."]
categorie = [c.categoria for c in lista_clienti]
categorie_counter = Counter(categorie)
print(f"distribuzione categorie clienti: {categorie_counter}")
print(f"categoria più frequente: {categorie_counter.most_common(2)}") # i 2 elementi + numerosi
print(f"totale: {categorie_counter.total()}")

vendite_gennaio = Counter({"Laptop": 13, "Tablet": 15})
vendite_febbraio = Counter({"Laptop": 3, "Stampante": 1})
vendite_bimestre = vendite_gennaio+vendite_febbraio
# aggregare informazione
print(f"Vendite Gennaio: {vendite_gennaio}")
print(f"Vendite Febbraio: {vendite_febbraio}")
print(f"Vendite bimestre: {vendite_bimestre}")
# fare la differenza
print(f"Differenza di vendite: {vendite_gennaio-vendite_febbraio}")
# modificare i valore in the fly
vendite_gennaio["Laptop"] += 4
print(f"Vendite Gennaio: {vendite_gennaio}")

# metodi da ricordare
# c.most_common(n) #restituisce gli n elementi più frequenti
# c.total() # somma dei conteggi
------------------------------------------------------------------------------
# DEQUE
coda_ordini = deque()
for i in range (1, 10):
       cliente = cr(f"Cliente {i}", f"cliente{i}@polito.it", "Gold")
       prodotto = pr(f"Prodotto{i}", 100.0*i)
```

```
        ordine = Ordine([RigaOrdine(prodotto, 1)], cliente)
        coda_ordini.append(ordine)
print(f"Ordini in coda: {len(coda_ordini)}")
while coda_ordini:
      ordine_corrente = coda_ordini.popleft()
      print(f"Sto gestendo l'ordine del cliente: {ordine_corrente.cliente}")
print(f"Ho processato tutti gli ordini!")
```

TdP 04 - graphical interfaces

FLET è una libreria che implementa flet in python per fare interfacce grafiche, per utilizzarla bisogna prima importarla (*import flet as ft*) a seguito di cui sarà presente un codice che interagisce con l'interfaccia grafica, questa porzione di codice viene inserita all'interno di un metodo *def main(page: ft.Page)* che presenta come argomento un oggetto di tipo pagina, il codice deve finire con una chiamata al metodo app *ft.app(target=main)* oppure al metodo .run per versione più recenti, in modo che la libreria automaticamente crea un'interfaccia vuota nel sistema operativo, detta pagina in cui possiamo scrivere quello che vogliamo.
Le app flet sono organizzate in containers (collection grafica), per cui flet organizza l'interfaccia grafica in gerarchie di container, dove la pagina è il container più generale (in cima), nella pagina si possono inserire altri container che avranno al loro interno altri elementi grafici.
Il container più generale è il view che associamo all'applicazione e che conterrà la nostra pagina, inoltre possiamo specificare dove visualizzare la nostra applicazione, tipicamente all'interno di una finestra del nostro sistema operativo che viene richiamata chiamando *ft.app(target=main, view=ft.AppView.FLET_APP)*, oppure all'interno di una finestra del browser chiamando *ft.app(target=main, view=ft.AppView.WEB_BROWSER)*.

Gli elementi che inseriamo nella pagina sono detti controls, delle classi python del modulo flet che bisogna instanziare all'interno della pagina e verranno chiamati come *nome = ft.controls(argomento)*, per aggiungere il controllo alla pagina si usa il metodo append *page.controls.append(nome_controllo)* e in seguito bisogna inserire il metodo *page.update()* per aggiornare tutti i controlli la pagina, oppure è possibile usare il metodo *page.add(nome_controllo)* che esegue entrambi i comandi.
Per aggiungere righe si usa il metodo *row = ft.Row([lista controlli])* che presenta come argomento una lista di controlli che vogliamo mettere in riga uno dopo l'altro.
Alcuni dei controlli che si possono inserire nell'interfaccia grafica sono interattivi, come i pulsanti che rappresenta il prototipo di controlli che possono essere associati a qualcosa che il codice deve fare, per cui nella definizione della classe pulsante bisogna passare il parametro onclick e il nome di una funzione che verrà chiamata quando dall'interfaccia grafica si clicca sul pulsante.
nota: documentazione https://flet.dev/docs/controls

ex: button

```
import flet as ft

def main(page: ft.Page):

    def handle_button(e):
        page.add(ft.Text("Ciao " + name.value + "!"))

    name = ft.TextField(label="Your name")
    page.add(
        ft.Row(controls=[
            name,
            ft.ElevatedButton(text="Say my name!", on_click=handle_button)
        ])
    )

ft.app(target=main)
```

ex: to do list

```python
import flet as ft
def main(page):

    def handleAdd(e):
        page.add(ft.CupertinoCheckbox
                 (label=txtBox.value, value=False))
        txtBox.value = ""
        txtBox.update()

    txtBox = ft.TextField(label="Write here something to remember", width=500)
    button = ft.ElevatedButton(text="Add",on_click=handleAdd)
    page.add(ft.Row([txtBox, button]))

ft.app(target=main)
```

ex: counter

```python
import flet as ft

def main(page: ft.Page):
    page.title = "TdP Example Counter"
    page.vertical_alignment=ft.MainAxisAlignment.CENTER

    def handleRemove(e):
        val.value = val.value-1
        val.update()
    def handleAdd(e):
        val.value = val.value+1
        val.update()

    val = ft.TextField(width=100,border_color="green",value=0,text_align="center")
    page.add(ft.Row([
        ft.IconButton(icon=ft.icons.REMOVE,icon_color="green",on_click=handleRemove),
        val,
        ft.IconButton(icon=ft.icons.ADD,icon_color="green",on_click=handleAdd)
    ],
    alignment=ft.MainAxisAlignment.CENTER
    ))
    page.update()

ft.app(target=main)
```

In generale i design patterns sono una serie di regole che ci diamo quando scriviamo il codice per rendere il codice più riutilizzabile e mantenibile.
Pattern MVC (Model-View-Controller) è un modo di organizzare il codice per delegare le varie funzionalità a diversi moduli, in particolare comanda di organizzare il codice in tre moduli, uno detto model che contiene la logica del programma (interazione e manipolazione dei dati), uno detto view che conterrà solo le definizioni di classi che definiscono e regolano il comportamento dell'interfaccia grafica, e uno detto controller che fa parlare il view con il model, per cui instanzierà una serie di metodi per gestire l'interfaccia grafica in funzione di quello che ci dice l'utente e di quello che ci dice il modello, quindi sarà presente una serie di metodi che leggeranno le informazioni fornite dall'utente, le passeranno al modello e aspetteranno l'output della richiesta da parte dell'utente.

Ciclo delle interazioni: definiamo una interfaccia grafica in cui chiediamo all'utente una serie di informazioni, il controller percepirà l'input dell'utente, lo salverà in delle variabili locali e lo passerà al modello, in seguito il modello effettua una elaborazione algoritmica fornendo un output che verrà passato all'interfaccia grafica, nello specifico alla classe controller, infine avverrà l'aggiornamento dell'interfaccia grafica per visualizzare il risultato della manipolazione.
In python saranno presenti tre classi: *view* che definisce tutti i controlli forniti da flet, *controller* che parla con la view, legge tutti gli input dell'utente e scrive sull'interfaccia grafica tutti gli output del modello, è inoltre responsabile di dialogare con il modello, per cui avrà dei metodi che effettuano una chiamata a qualche metodo del modello per recuperare le informazioni che gli servono, e *model* per la gestione dei dati.
file main:

```python
import flet as ft
from controller import Controller
from view import View

def main(page: ft.Page):  1 usage
    v = View(page)
    c = Controller(v)
    v.setController(c)
    v.caricaInterfaccia()

ft.app(target=main)
```

file controller:

```python
from view import View
from model import Model
import flet as ft
class Controller(object):  2 usages
    # contiene solo i metodi che interaisco con il view e con il modello
    def __init__(self, view: View):
        self._view = view
        self._model = Model()
```

file model:

```python
class Model(object):  2 usages
    #contiene la parte di interazione con i dati
    def __init__(self):
        pass
```

file view:

```python
import flet as ft                                                
```

I database sono lo strumento che utilizziamo per avere dei dati salvati permanentemente da qualche parte, per cui vogliamo che le nostre applicazioni non fossero vuote ogni volta che vengono lanciate, ma che possano recuperare dei dati a partire da una memoria persistente che possiamo usare per recuperare/modificare/aggiungere/leggere dati.
Per accedere a questi database da python si utilizza una libreria, mySQL-connector.
nota: documentazione https://dev.mysql.com/doc/connector-python/en/connector-python-example-cursor-select.html
nota: tutte le librerie -connector sono una serie di metodi e classi che permettono di interagire con un server

Un DBMS (database management system) è un software che ci permette di definire/creare/mantenere e controllare l'accesso a un database, per cui utilizzeremo lo standard MySQL e delle librerie che si interpongono tra DBMS (specifiche ad esso) e utente, nel nostro caso utilizzeremo la libreria mySQL-connector.
La libreria che si utilizza deve rispettare lo standard pep-0249 che stabilisce quali contenuti una libreria di tipo connector ad un database deve offrire, nel particolare tutte le librerie devono offrire due strumenti detto Connection e Cursor.
nota: come i DBMS implementano lo standard https://troels.arvin.dk/db/rdbms/

mySQL-connector può essere installata attraverso il comando pip install mysql-connector-python sul terminale a livello di sistema, oppure è possibile installarlo nel virtual environment come con flet.
Connection è una funzione che ci permetta di stabilire una connessione con il database, per cui offre un metodo **connect()** che serve per creare una connessione con il database e prende come argomento almeno quattro parametri, per cui bisogna specificare sempre username e password, l'indirizzo dove il database è hostato, nel nostro caso sarà sempre 127.0.0.1 che rappresenta il local host, e quale database si vuole andare ad aprire, quindi l'oggetto che viene creato è un oggetto di tipo mysql-connection e rappresenta un'istanza di una classe che viene utilizzata per aprire e gestire la connessione ad un database.
nota: la connessione al database è una risorsa quindi dopo averla usata deve essere chiusa attraverso il comando ***cnx.close()***, in quanto altrimenti rimarrebbe aperta
cnx = mysql.connector.connect(user = 'username', password = 'password', host = '127.0.0.1', database = 'db')

L'apertura di una connessione può creare delle eccezioni, ovvero può fallire e i modi in cui può fallire devono essere gestire, in particolare potremmo avere tre tipi di errori, tra cui ACCESS_DENIED_ERROR per cui sono stati sbagliati la password e/o l'username, BAD_DB_ERROR per cui il database non esiste, errore di quando il server è spento, per cui bisogna trovare il server tra i servizi e aprire il server, quindi tipicamente la creazione di una connessione viene messa in un try-catch che prova a creare una connessione e se fallisce rende esplicito il tipo di errore, alla fine bisogna mettere connection.close().
Per motivi di sicurezza è possibile creare una connessione passandogli un file, in cui sono inseriti i campi user, password, host, database e raise_on_warnings=True, utilizzando il comando ***cnx = mysql.connector.connect(option_file = '/ect/mysql/connectors.cnf'***
nota: all'esame bisogna andare a pescare questo file e modificare la password ed eventualmente il nome del database

```python
import mysql.connector
from mysql.connector import errorcode

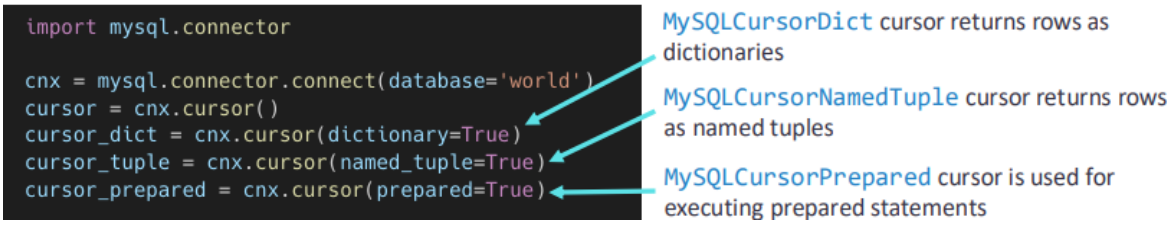
try:
  cnx = mysql.connector.connect(user='scott',
                                database='employ')
except mysql.connector.Error as err:
  if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
    print("Something is wrong with your user name or password")
  elif err.errno == errorcode.ER_BAD_DB_ERROR:
    print("Database does not exist")
  else:
    print(err)
else:
  cnx.close()
```

```
[client]
user=John
password=Wick
host=127.0.0.1
database=tests
raise_on_warnings=True
```

La connessione è una relazione che il codice python ha con il DBMS e data questa connessione possiamo interagire con il DBMS in diversi modi Create/Update/Delete/Read data tramite delle query SQL detti statement che devono essere passati al server tramite una struttura di handle detta **cursor**.
Il cursor funziona come un puntatore, ovvero funziona come una variabile che punta ad un indirizzo di memoria e scorrerà su tutti di dati della tabella, per cui nel momento in cui il cursore eseguirà una query punterà ad una riga del

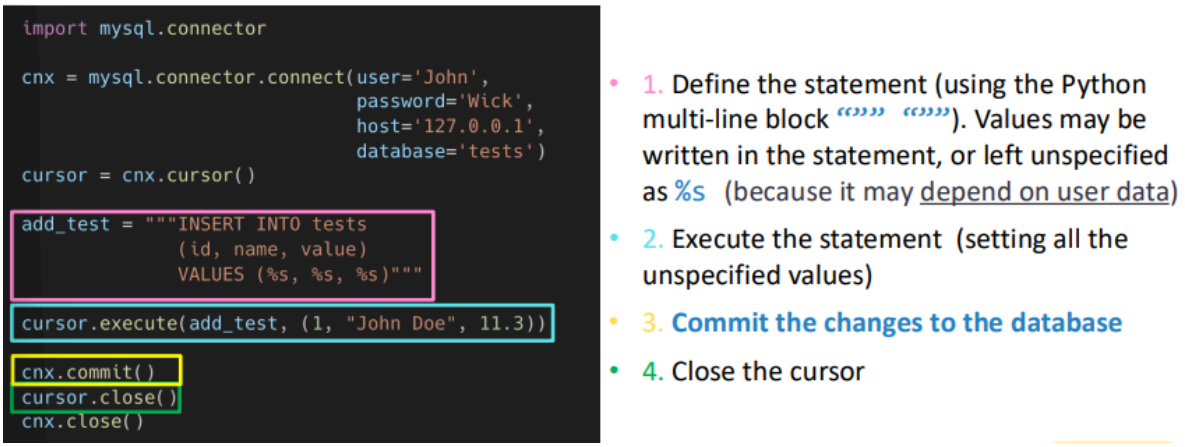
database e noi potremmo leggere tutte le righe una dopo l'altra (si inizia a leggere a partire della riga successiva) presenti nel result set.

Il cursore viene creato a partire dalla connessione tramite il metodo ***cnx.cursor()*** della connection, per cui il cursore è un oggetto primitivo del tipo MySQLCursor, ovvero un oggetto da cui altri oggetti ereditano, ed è un oggetto che può eseguire delle query per cui noi possiamo recuperare i dati dal cursore.
Esistono diversi altri cursori che ereditano la cursor e che possiamo invocare passando un argomento a cursor, questi diversi tipi di cursori differiscono solo per come i risultati vengono restituiti, tra cui *cnx.cursor(dictionary=True)* che restituisce una lista di dizionari, *cnx.cursor(named_tuple=True)* che restituisce una lista di tuple con nome, invece se non viene specificato niente i risultati vengono fuori come una lista di tuple semplici



Per usare il cursore bisogna scrivere una query, ovvero una stringa di testo che esegue delle istruzioni in SQL, che viene passata al metodo ***cursor.execute(query)*** del cursore, andando ad inserire in cursor il result set di quella query, ovvero la tabella risultante.
*query = "Select * from clienti where mail = %s"*
cursor.execute(query, (cliente.mail))
La query può dipendere da input forniti dall'interfaccia grafica, per cui solitamente avremmo dei textfield in cui l'utente scriverà qualcosa e noi dovremmo modificare la query in funzione dell'input dell'utente, quindi bisogna usare delle query parametriche, dette parametric statements, ovvero delle query che abbiano dei parametri dentro di cui si conosce il tipi e che andranno ad essere modificati.
Tutti gli statement che scriviamo devono essere parametrici, per cui in fase di creazione viene definita la sintassi SQL (modello) che presenta dei segnaposto per le quantità variabili (parametri), che verranno definire nella fase di esecuzione prima di eseguire l'istruzione, dopo eventualmente aver fatto dei controlli.
Data una query di questo tipo i parametri vengono automaticamente convertiti in funzione dei loro tipi primitivi, inoltre verranno ignorati se contengono dei caratteri pericolosi.

Prototipo del codice per INSERT, UPDATE and DELETE statements:



nota: i parametri %s indica che qualcuno deve passare un parametro li, indipendentemente dalla query per eseguirla bisogna usare il metodo execute a cui viene passata la query testuale e una tupla di parametri ***cursor.execute(query, (tupla parametri))***
nota: tutte le query che modificano il database devono essere seguite da un commit altrimenti le modifiche non vengono eseguite dal server ***cnx.commit()***

```python
import mysql.connector

cnx = mysql.connector.connect(user='John',
                              password='Wick',
                              host='127.0.0.1',
                              database='tests')
cursor = cnx.cursor()

update_test = """UPDATE tests
                 SET value = %s
                 WHERE id = %s"""

cursor.execute(update_test, (99.9, 3))

cnx.commit()
cursor.close()
cnx.close()
```

```python
import mysql.connector

cnx = mysql.connector.connect(user='John',
                              password='Wick',
                              host='127.0.0.1',
                              database='tests')
cursor = cnx.cursor()

delete_test = """DELETE FROM tests
                 WHERE id = %s"""

cursor.execute(delete_test, (5,))

cnx.commit()
cursor.close()
cnx.close()
```

Prototipo del codice per lettura del database: creo una connessione con *mysql.connector.connet()*, creo un cursore con *connection.cursor()*, scrivo la query che tipicamente sarà del tipo SELECT qualcosa FROM tabella WHERE una serie di clausole, viene eseguita la query con *cursor.execute(query)*, tipicamente le query che leggono hanno un pezzo di codice che fa qualcosa sul cursore, quindi tipicamente si utilizza un ciclo sul cursore che prende tutte le righe (tipicamente tuple) per poi poterci elaborare.
nota: usiamo il cursore come se fosse un iterable, ovvero come se fosse una lista

```python
import mysql.connector

cnx = mysql.connector.connect(user='John',
                              password='Wick',
                              host='127.0.0.1',
                              database='prova')
cursor = cnx.cursor(dictionary=True)
query = """SELECT * FROM test"""
cursor.execute(query)

for row in cursor:
    print(row["id"], row["name"], row["value"])

cursor.close()
```

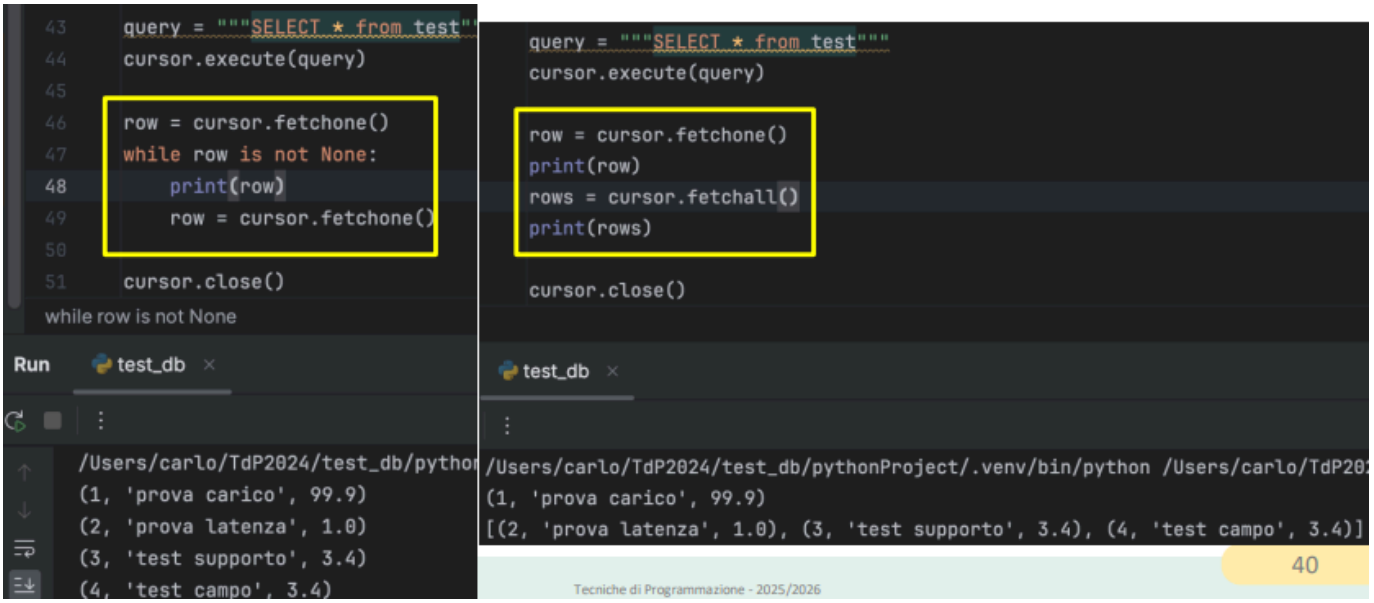
- If we use a `MySQLCursorDict` the results set is read as a dictionary, so we can iterate through the data accordingly

La query tira fuori una tabella detta result set e il cursore si piazza in cima alla tabella e quando cicliamo il cursore inizia a scorrere da una riga alla successiva prendendo i valori nella tupla di quella riga, per cui i dati sono disponibili una lista alla volta ed è possibili leggere questi elementi solo ciclando.

La libreria ci offre tre metodi comodi per leggere senza dover ciclare:

- *fetchone()* recupera la riga successiva del result set di una query e restituisce una singola sequenza, andando avanti con l'iteratore finché non sono presenti altri righe, per cui restituisce None se non ci sono più righe disponibili
- *fetchmany(N)* recupera il set successivo di N righe del risultato di una query e restituisce una lista di N tuple (o dizionari o tuple con nome, se si utilizzano altri cursori specializzati)
- *fetchall()* recupera tutte le righe (o tutte quelle rimanenti) del result set di una query e restituisce una lista di tuple (o dizionari o tuple con nome, se si utilizzano altri cursori specializzati), se non ci sono più righe disponibili restituisce una lista vuota

nota: è quindi importante non mischiare i metodi
nota: quando si esegue una query per leggere i dati, è necessario gestire tutti i risultati, in quanto altrimenti quando si prova a chiudere il database senza aver letto tutti i risultati solleva un errore del tipo *msql.connector.errors.InternalError: Unread result found*, per cui quando si prova a chiudere un cursore che è posizionato non alla fine di un result set allora viene sollevato un errore, per cui è meglio usare *fetchall()*



I tipi di SQL che vengono generati nel result set vengono automaticamente mappati sul rispettivo python, per evitare questa conversione è possibile passare al cursore il parametro raw=true (non faremmo mai)

TdP 5.1 - pattern DAO

Il pattern DAO (data access object) è una serie di regole nell'interagire con un database, per cui l'obiettivo è incapsulare l'accesso al database in una o più classi separate e distanza dal modello e dalla parte grafica, quindi avremmo un accesso al database gestito da un modulo che contiene una classe chiamata DAO, che è totalmente indipendente dai bisogni dell'applicazione e potenzialmente ri-utilizzabile da altre applicazioni.
DAO è un pattern di programmazione che ci serve per astrarre la relazione tra il database e il programma, nel DAO sarà presente una selezione di metodi che si occuperà di aggiungere, leggere, eliminare e modificare i dati, per cui il modello non deve sapere che dietro c'è un database e come noi interagiamo con il database, per cui tipicamente il modello farà un import del DAO, creerà un'istanza del DAO e poi userà i metodi del DAO.
Quindi saranno presenti due tipologie di classi: classi client che rappresenta il modello, ovvero classi dell'applicazione che hanno bisogno di accedere al database, ma ignoranti dei dettagli del database, e classi DAO che raccolgono tutto il codice che fa accesso al database e risultano essere ignoranti dell'obiettivo dell'utente finale.

Il DAO tipicamente non ha uno stato, quindi tipicamente il __init__ sarà vuoto, l'unica cosa che potrebbe avere è la connection, il DAO gestisce solo un tipo di dato, quindi avremmo una DAO per ogni database che useremo, inoltre di solito deve offrire dei metodi CRUD ovvero metodi per creare, leggere, aggiornare ed eliminare dati, spesso offrirà anche dei metodi di ricerca restituendo una collezione di DTO.
Per implementare il pattern DAO bisogna avere due cose, delle classi di basso livello che fanno la connessione al database e delle classi DTO (Data Transfer Object) che sono delle classi specifiche per raccogliere dati e vengono implementate tramite @dataclass, per cui per ogni tabella del database sarà presente una dataclass i cui argomenti sono le colonne della tabella, per cui quando si legge dalla tabella verranno create delle istanze della dataclass.
Per rispettare il pattern DTO dovremmo creare delle classi di tipo dataclass contenenti le informazioni del singolo che dovranno avere una funzione di ___hash__ che per convenzione indichiamo che due istanze sono lo stesso oggetto se la chiave primaria del database è la stessa, stessa cosa per la funzione __eq__

creazione database su DBeaver: localhost → databases doppio click → create new database → nome database
creazione query su DBeaver → tasto SQL in alto a sinistra → seleziona database da menu a tendina → scrivere query → per eseguirla secondo tasto a destra (il secondo triangolo)
una volta soddisfatti della query creata su DBeaver la possiamo inserire all'interno do cursor.execute(query) su pycharm
flusso logico da seguire: vedere che tabelle sono contenute nel database → per ogni tabella costruire una classe di tipo dataclass che deve presentare una funzione di __hash__ e __eq__ che usano la chiave primaria della tabella e una funzione di __str__ per descrivere i parametri

esempio classe dataclass:

```python
from dataclasses import dataclass
@dataclass  10 usages
class ClienteRecord:
    name: str
    mail: str
    categoria: str
    def __hash__(self): #hash della chiave primaria
        return hash(self.mail)
    def __eq__(self, other): #sono uguali se la chiave primaria è uguale
        return self.mail == other.mail
    def __str__(self):
        return f"{self.name} -- {self.mail} ({self.categoria})"
```

esempio funzione in classe dao per lettura:

```python
def getAllProdotti(self):  1 usage
    cnx = mysql.connector.connect( #creo connessione
        user="root",
        password="", #XAMPP che non presenta password, altrimenti devo mettere la password della connessione
        host="127.0.0.1", #indica local host
        port=3307, #devo specificare la porta in quanto diversa da deafult 3306
        database="sw_gestionale")
    cursor = cnx.cursor(dictionary=True) #creo cursore che restituisce dizionari
    cursor.execute("Select * from prodotti") #eseguo la query
    row = cursor.fetchall() #restituisce una lista di dizionari
    res = [] #lista vuota dove vado a mettere i risultati
    for p in row: #ciclo sulla tabella ottenuta come result set
        res.append(ProdottoRecord(p["nome"], p["prezzo"])) #inserire i nomi presenti nel database
    #finita l'interazione ricordati di chiudere tutto
    cursor.close()
    cnx.close()
    return res
```

esempio funzione dao per scrittura:

```python
def addProdotto(self, prodotto):
    cnx = mysql.connector.connect(
        user="root",
        password="",
        host="127.0.0.1",
        port=3307,
        database="sw_gestionale")
    cursor = cnx.cursor()
    query = "insert into prodotti (nome, prezzo) values (%s, %s)"
    # query per inserire nuova riga nella tabella prodotti
    cursor.execute( query, params: (prodotto.name, prodotto.prezzo_unitario))
    cnx.commit() #istruzione che va a modificare il database
    # finita l'interazione ricordati di chiudere tutto
    cursor.close()
    cnx.close()
    return
```

esempio funzione su modello:

```python
import random
from collections import deque, Counter, defaultdict
from dao.dao import DAO
from gestionale.core.cliente import ClienteRecord as cr
from gestionale.core.prodotto import ProdottoRecord as pr
from gestionale.vendite.ordini import Ordine, RigaOrdine

class GestoreOrdini:  3 usages
    def __init__(self):
        self._ordini_da_processare = deque()
        self._ordini_processati = []
        self._statistiche_prodotti = Counter()
        self._ordini_per_categoria = defaultdict(list)
        self._dao = DAO() #nel modello ti devi creare un'istanza del DAO
        self._allP = []
        self._allC = []
        self._fill_data()

    def _fill_data(self):  1 usage
        #leggo prodotti e clienti dal DB e creo degli ordini randomici per testare l'app
        self._allP.extend(self._dao.getAllProdotti()) #se allP non è vuota vado ad aggiungere
        self._allC.extend(self._dao.getAllClienti())
        for i in range(10):
            indexP = random.randint( a: 0, len(self._allP)-1)
            indexC = random.randint( a: 0, len(self._allC)-1)
            ordine = Ordine([RigaOrdine(self._allP[indexP],random.randint( a: 1,  b: 5))],
                            self._allC[indexC])
            self.add_ordine(ordine)
```

nota: per usare le funzioni presenti nel dao bisogna creare un'istanza del dao nel modello all'interno della funzione __init__ come *self._dao = DAO()* e importarla

Per evitare di creare una connessione per ogni funzione della classe dao è possibile creare una classe all'interno del package dao, chiamata DBConnect, in cui creiamo la connessione in modo da centralizzare in un solo punto tutti i parametri *def getConnection(cls)*, in seguito chiamo il metodo creato all'interno del dao invece di creare una connessione ad ogni metodo *cnx = DBConnect.getConnection()*.
In questo modo sto creando diverse istanze della classe DBConnect, che tuttavia risulta essere una risorsa scarsa di cui vorrei centralizzare l'utilizzo e chi la crea la risorsa, per cui viene utilizzato un pattern di programmazione detto singleton che implica che quando si crea una classe non è possibile creare diverse istanze della classe, ma solo una, per implementare ciò si utilizza il decoratore @classmethod sul nome del metodo, quindi non sarà più un metodo dell'istanza, ma un metodo della classe, per cui non verrano create diverse istanze della classe, ma si usa direttamente il metodo come metodo di classe.

dbConnect.py × dao.py

```python
import mysql.connector
class DBConnect:  6 usages  new *
    @classmethod  5 usages  new *
    def getConnection(cls):
        try:
            cnx = mysql.connector.connect(
                user="root",
                password="", #password="rootroot"
                host="127.0.0.1",
                port=3307, #non usata dal prof
                database="sw_gestionale")
            return cnx
        except mysql.connector.Error as err:
            print("non risco a collegarmi al db")
            print(err)
            return None
```

TdP 5.2 - ORM

La relazione fra le entità del database e gli oggetti nel software python sono gestite tramite il pattern di programmazione ORM (Object Relational Mapping) che ci fornisce una serie di regole per trasferire dati dal database al software.

Il database è formato da una serie di tabelle ognuna delle quali indicherà un oggetto di tipo diverso, per cui le diverse istanze dell'oggetto rappresentano le diverse righe della tabella, per mappare questo rappresentiamo le entità come oggetti di tipo DTO, tuttavia le tabelle presentano anche delle relazioni che vengono mappate attraverso delle foreign keys all'interno delle tabelle che presentano dati oppure tramite una terza tabella di relazione, che dovranno essere mappate in python.
nota: le tabelle nel database presentano nome al plurale, mentre le classi DTO avranno lo stesso nome al singolare e delle property che avranno lo stesso nome delle colonne della tabella

Creazione sistematica dei DTO:

Tables → data class ORM rules:

1) Crea una dataclass per ogni entità principale del database, ovvero solo per tabelle che contengono dati, quindi bisogna escludere le tabelle utilizzate per memorizzare relazioni n:m
   i nomi delle classi dovrebbero corrispondere ai nomi delle tabelle al singolare (Utente; User)
2) La classe dovrebbe avere un attributo per ogni colonna della tabella, con nomi corrispondenti
   – Secondo le convenzioni di denominazione di Python (NUMERO_DATI -> numero_dati)
   – Corrispondenti al tipo di dato
   – Escluse le colonne utilizzate come chiavi esterne
3) Aggiungi i metodi getter (@property) e setter (@attr.setter) per gli attributi, se necessario. Il metodo setter non può essere specificato se la dataclass utilizza il parametro frozen=True.
4) Definisci __eq__() e __hash__() utilizzando l'esatto insieme di campi che compongono la chiave primaria della tabella, se sono presenti più chiavi primarie allora bisogna usare una tupla
   nota: all'esame è fondamentale chiedersi quale tabelle e quali attributi delle varie tabelle ci servono
5) Relationships, Foreign keys → Class: definisci attributi aggiuntivi nelle classi DTO per ogni relazione che vogliamo gestire facilmente all'interno della nostra applicazione, il come dipende dal tipo di relazione
   – Non necessariamente *tutte* le relazioni!

Esistono due classi di relazioni, la più semplice è la relazione con cardinalità 1 che corrisponde a un attributo che fa riferimento all'oggetto Python corrispondente, non al valore della chiave primaria, ovvero per introdurre una relazione fra due classi di tipo 1:1 si introduce in una classe un oggetto di tipo pari a quello dell'altra classe (o viceversa).
nota: utilizzare nomi singolari

1:1 relationship

```
STUDENTE                        PERSONA
--------                        -------
matricola (PK)                  codice_fiscale (PK)
fk_persona                      fk_studente

@dataclass                      @dataclass
class Studente:                 class Persone:
      persona: Persona                studente: Studente
      codice_fiscale: str             matricola: int
```

Per le relazioni con cardinalità N si mappa un elemento di una tabella ad una collezione dal lato molti, per cui gli elementi della collezione (ad esempio lista o set) sono oggetti Python corrispondenti (non valori di chiave primaria), inoltre la classe classe dovrebbe avere metodi per leggere (get, ...) e modificare (add, ...) la collezione
nota: utilizzare sostantivi plurali
nota: in python possiamo avere un'esplicita relazione in entrambe le direzioni in funzione dello scopo dell'applicazione aggiungendo un attributo ad entrambe le tabelle, mentre in SQL vengono mappate come tabelle extra

1:N relationship

```
STUDENTE                        CITTA
------------------              --------------
matricola (PK)                  cod_citta (PK)
fk_citta_residenza              nome_citta

@dataclass                      @dataclass
class Studente:                 class Citta:
      cittaResidenza: Citta           studentiResidenti: list[Studente]
```

N:M relationship

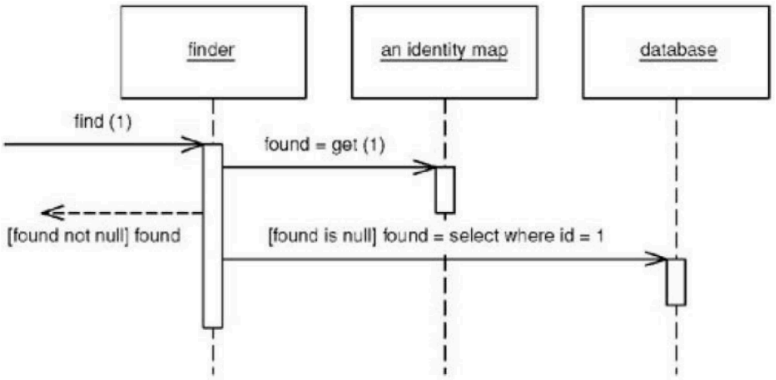
```
ARTICLE                 AUTHORSHIP              CREATOR
---------------         ---------------         ---------------
id_article (PK)         id_article (FK,PK*)     id_creator (PK)
Article data…           id_creator (FK,PK*)     Creator data…
                        id_authorship (PK#)

@dataclass                                      @dataclass
class Article:                                  class Creator:
      creators: set[Creator]                          articles: set[Article]
```

All'inizio dell'implementazione del nostro algoritmo dobbiamo compiere una scelta:

- creare le relazioni riportando la foreign key, ovvero riportando la chiave primaria dell'oggetto con cui ci relazioniamo, questo risulta essere più semplice nella creazione degli oggetti, tuttavia quando bisogna utilizzare tale relazione allora sarà necessario fare una query apposita per recuperare il dato di cui si ha bisogno
  id_citta_residenza: int
- quando si crea l'istanza dell'oggetto, si crea anche l'istanza del secondo oggetto con cui il primo oggetto ha una relazione e lo si inserisce dentro, per cui nel momento in cui si crea l'oggetto esso presenta già tutto quello che ci serva, tuttavia la query di creazione sarà più complicata in quanto bisogna fare un join di più tabelle
  non è detto che i dati poi vengano usati, quindi bisogna chiedersi se queste relazioni sono necessarie per la nostra applicazione
  citta_residenza: Citta
- utilizzare un metodo lazy, per cui ci creiamo un oggetto che contiene un attributo del secondo oggetto senza andarlo davvero a leggerlo, in quanto viene inizializzato a None oppure ad un oggetto vuoto, per cui l'attributo viene lasciato vuoto e andando a leggere dal dao quell'oggetto quando mi serve solo una volta
  citta_residenza : Citta = None // lazy
  citta_residenza : Citta = field(default_factory=lambda: []) // lazy

Esiste un problema legato all'identità, in quanto può accadere che un singolo oggetto venga recuperato molte volte all'interno di query differenti, soprattutto in relazioni di tipo N:M, ma lato codice python non vogliamo avere diversi oggetti aventi tutti stessi campi, per evitare di creare oggetti pseudo identici (oggetti con nomi diversi, ma stessa chiave primaria) è possibile andare a salvare in una mappa usando un dizionario tutti gli oggetti che leggiamo, per cui prima di creare un nuovo oggetto bisogna chiedere alla mappa se è già esistente, quindi se è già presente nella mappa prendiamo quell'oggetto al posto di fare una nuova query, mentre se l'oggetto non viene trovato nella mappa allora delego l'operazione al database, facendo una query e inserendo poi l'oggetto nella mappa (pattern identity map).



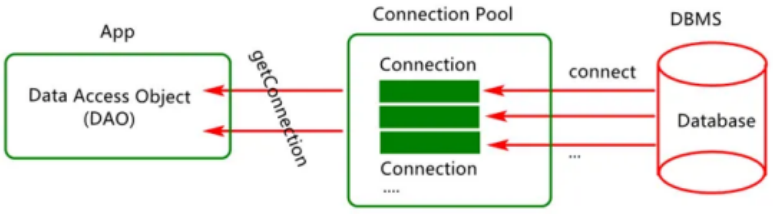
Tipicamente avremmo un identity_map per ogni tabella, dove la mappa rappresenta un dizionario avente come chiave una serie di campi che rappresentano la chiave primaria della tabella e come valore l'oggetto stesso che rappresenta la tabella.
Utilizzo della Identity Map:

1) crea e memorizza la identity_map nel Model
2) passa un riferimento della identity_map ai metodi del DAO
3) nel DAO, quando carichi un oggetto dal database, controlla prima la mappa
   – Se esiste un oggetto corrispondente, terminalo e restituiscilo (senza crearne uno nuovo)
   – Se non esiste un oggetto corrispondente, crea un nuovo oggetto e inseriscilo nella mappa per utilizzi futuri
4) se possibile, controlla la mappa prima di eseguire la query

TdP 5.3 - connection pooling

Per rendere più efficiente il codice, al posto di creare una connessione per ogni query che facciamo, è possibile creare una o più connessioni all'inizio quando lanciamo il codice che restano in attesa aspettando che qualcuno le usi, per cui bisogna riscrivere il DBConnect in modo tale che non debba creare una connessione nuova ogni volta che viene chiamato, ma che ne presenti un n numero a disposizione e che ne restituisca una delle n quando chiamato, quindi il

DAO prende in prestito una connessione al posto di crearla e quando si esegue il comando cnx.close() non si chiude la connessione, in quanto viene restituita al pool.



mysql connector implementa già il pooling tramite il modulo mysql.connector.pooling che aprirà diverse connessioni e gestirà il prestito delle connessioni a chi le chiede, per cui deve presentare diverse proprietà rispetto alla connessione singola, tra cui la dimensione del pool, ovvero quante connessioni in parallelo riesce a gestire, e un nome.
Per ogni richiesta di connessione, il pool fornisce la prima connessione disponibile, quindi se un pool è esaurito, viene sollevata un'eccezione di tipo PoolError.
Il codice può presentare più pool che si collegano a database diversi, in quanto il codice può avere più dao che interagiscono con più database.

Per creare un pool si usa l'istruzione *cnxpool = mysql.connector.pooling.MySQLConnectionPool()* che crea un'istanza dell'oggetto MySQLConnectionPool che implementa il pooling, il costruttore presenta una serie di parametri, tra cui il nome della pool, il numero di connessioni che possiamo creare in parallelo e poi i parametri che siamo abituati a chiamare per creare una connessione.
In seguito bisogna utilizzare il metodo *cnx = get_connection()* che restituisce una connessione presente nel pool, in seguito deve essere restituita al pool usando il metodo close().

installare database su DBeaver
tasto destro sul nome progetto → open in → explorer → cartella database → file.sql → trascinalo dentro DBeaver (contiene le istruzioni per creare il database) → esegui file (terzo tasto arancione avente forma di foglio di pergamela con un play dentro) → nota: controlla di eseguire play dentro il localhost → finita operazione fare tasto destro sul nome localhost → refresh → crea nuova riga con nome database installato

TdP 06 - ricorsioni

Una definizione ricorsiva è quella in cui il termine definito compare nella definizione stessa, per cui un metodo è ricorsivo quando all'interno della definizione del metodo (la funzione richiama se stessa), il metodo stesso viene richiamato direttamente o indirettamente, quindi un algoritmo che utilizza metodi ricorsivi viene detto algoritmo ricorsivo.
La ricorsione ha una struttura if/else, dove l'else è la condizione non terminale che definisce il modo in cui si va a dividere il problema in sottoproblemi (growing call stack), mentre l'if è la condizione terminare che indica quando si è raggiunto il problema più piccolo che in seguito viene risolto (unwinding call stack), per evitare di entrare in un loup python ha instaurato una failsafe che dopo tot iterazioni della funzione restituisce un errore del tipo RecursionError, per cui bisogna sempre scrivere una condizione terminale.
Bisogna mantenere uno stato di esecuzione per far comunicare le funzioni del call stack, tipicamente come input/output della funzione, oppure è possibile incapsulare il metodo ricorsivo in una classe avente un attributo privato in cui viene messo il risultato.

Ogni programma ricorsivo ammette una implementazione iterativa, la strada da seguire dipende dal tipo di problema, per cui è bene implementare la soluzione più facile e sicura da scrivere, inoltre esistono delle tipologie di problemi in cui la ricorsione si testa bene, tra cui problemi di divide et impera, in cui è presente un macro problema complesso che può essere diviso in problemi più semplici facilmente risolvibili, systematic exploration/enumeration (ex: percorso di costo minore), handling recursive data structures.
Divide et Impera: split a problem P into {Qi } where Qi are still complex, yet simpler instances of the same problem → solve {Qi }, then merge the solutions → merge & split must be "simple"

Exploration: bisogna andare ad enumerare tutte le possibili soluzioni di un determinato → solutions (built stepwise) – Paths – Permutations – Combinations → divide et Impera by “dividing” the possible solutions

```
def solve (problem):
    if is_trivial(problem):
        solution = solve_trivial(problem)
        return solution
    else:
        sub_problems = divide(problem)
        sub_solutions = []
        for sub_problem in sub_problems:
            sub_solutions.append(solve(sub_problem))
        solution = combine(sub_solutions)
        return solution
```

check termination

do recursion

- Explore (**S**) {
  - List<Step> steps = PossibleSteps (Problem, **S**) ;
  - for (each **p** in steps) {
    - **S**.Do (**p**)
    - Explore (**S**) ;
    - **S**.Undo (**p**) ;
  - }
- }

In caso di problemi grandi è necessario considerare l'efficienza del metodo ricorsivo, per cui per aumentare l'efficienza bisogna usare strutture dati appropriate, saltare i thread di ricorsione che non producono risultati (può portare a miglioramenti enormi), memorizzare nella cache i risultati intermedi se il sottoproblema corrispondente si ripresenta più volte (i miglioramenti dipendono dal problema ed è previsto un costo in termini di memoria).
Il pacchetto functools implementa funzionalità di caching che consentono la memorizzazione come *from functools import lru_cache* e *@lru_cache(maxsize=None)* da inserire prima della definizione del metodo, dove @lru_cache è un decoratore che avvolge una funzione in un oggetto chiamabile con meccanismo di memoization, che memorizza fino a un massimo di `maxsize` chiamate recenti, attraverso un dizionario avente come chiave la chiamata alla funzione (inclusi gli argomenti) e come valore il risultato della funzione.
nota: gli argomenti della funzione devono essere hashable affinché il decoratore funzioni, inoltre la cache LRU dovrebbe essere utilizzata solo quando si desidera riutilizzare valori calcolati in precedenza

```
from functools import lru_cache

@lru_cache(maxsize=None)
def recursion(problem, …):
    # do operations
    return result
```

Quicksort è un algoritmo di sorting, utilizzato per per ordinare liste/vettori, basato sul principio di divide et Impera, per cui viene scelto l’elemento pivot e dividere la lista in due sottoliste, in elementi minori dell'elemento pivot e in elementi maggiori dell'elemento pivot, e infine applica ricorsivamente l'algoritmo Quicksort alle sottoliste.
L'efficienza dell'algoritmo Quicksort dipende dalla scelta del pivot utilizzato per partizionare la lista, se non sono presenti informazioni a priori sulla lista si utilizza come pivot il primo elemento della lista.
La ricerca dicotomica presenta in input un vettore ordinato e dato un elemento x bisogna stabilire se il valore è contenuto nella lista, per cui viene divisa la lista in due metà e confrontare l’elemento centrale con x, in seguito si applica nuovamente il procedimento a una delle due metà (sinistra o destra, a seconda del risultato del confronto), per cui l’altra metà della lista può essere ignorata dato che il vettore è ordinato.

```
def recursion(…, level):
    // E - instructions that should be always executed (rarely needed)
    do_always(…)

    // A
    if terminal_condition:
        do_something(…)
        return …
    else:
    for … //a loop, if needed
        //B
        compute_partial()

        if filtro: //C
            recursion(…, level+1)

        //D
        back_tracking
```

TdP 07 - grafi

Un grafo è un oggetto anagrafico definito come un'insieme di due elementi cardine (nodi e archi), dove un grafo semplice è un grafo in cui è presente un solo arco che connette due nodi, un multigraph presenta più di un arco tra dei nodi, un loops è un nodo che è connesso a se stesso, uno pseudograph è un multigraph con loops.
I grafi possono essere non diretti, per cui non è presente un orientamento (se A è connesso a B, allora anche B è connesso ad A), orientati, per cui gli archi possono avere una sola direzione (se A è connesso a B, B non è connesso ad A), diretti, per cui un arco può avere una o due direzioni (o in un verso, o nell'altro, o entrambi), o network, ovvero grafo orientato in cui ad ogni arco viene associato un peso.
A Directed Graph (di-graph) G is a pair (V,E), where V is a (finite, ma non ordinato) set of vertices and E is a (finite, ma non ordinato) set of edges, that identify a binary relationship over V ($E \subseteq V \times V$).
An Undirected Graph is still represented as a tuple G=(V,E), but the set E is made of non-ordered pairs of vertices, per cui non importa l'ordine con cui viene scritta la coppia di archi (non abbiamo delle frecce, ma delle linee).
In un grafo non orientato il grado di un vertice (degree) è il numero di archi incidenti, ovvero a quanti altri nodi è connesso, in un grafo orientato bisogna distinguere tra il grado entrante che è il numero di archi in entrata, il grado uscente che è il numero di archi in uscita, mentre il grado è la somma del grado entrante e del grado uscente.
nota: i loops possono essere considerati sia come nodi entranti che come nodi uscenti
nota: un vertice con grado 0 è isolato

Il percorso è una collezione di nodi ordinata, per cui si definisce un percorso su un grafo G=(V,E) se esiste una sequenza ordinata di nodi tale che i vertici e gli archi appartengono al grafo e tali per cui esiste sempre un arco tra il nodo-iesimo e il nodo-iesimo + 1; su percorsi è possibili definire la lunghezza del percorso come il numero di archi che il percorso attraversa o uno in meno al numero di nodi (n-1).
Il ciclo è un percorso in cui il nodo iniziale e il nodo finale coincidono, per cui un grafo senza cicli viene detto acyclic, per cui è un grafo in cui a partire da un nodo non esiste alcun percorso tale per cui si possa ritornare al nodo di partenza, quindi è possibile definire la lunghezza del ciclo come il numero di archi o nodi percorsi.

Considerando i grafi non diretti è possibile definire un grafo connesso, se dati qualunque due vertici è possibile trovare sempre un percorso tra di loro per tutte le possibili coppie di vertici del grafo, se questa condizione non è vera per tutto il grafo è possibile individuare delle sezioni dette componenti connesse in cui vale questa condizione.
Considerando i grafi diretti è possibile definire un grafo fortemente connesso, se per ogni coppia ordinata di vertici (v, v'), esiste almeno un cammino che collega v a v'.
Un grafo si definisce completo se data ogni coppia di nodi è possibile trovare un percorso, per cui per ogni coppia di nodi è presente un arco che li connette (ogni nodo è connesso a tutti gli altri nodi del grafo), quindi in un grafo completo con n vertici, il numero di archi è $n(n-1)$ (no loops e diretto), $n(n-1)/2$ (no loops e non diretto), n^2 (loops e diretto), $n(n+1)/2$ (loops e non diretto).
La densità di un grafo G=(V,E) è dato dal rapporto tra il numero di archi del grafo e il massimo numero possibili di archi (archi del grafo completo): $d = |E(G)| / |E(K_{|V(G)|})|$

Un grafo non diretto aciclico è detto foresta, mentre un grafo non diretto aciclico e connesso è detto albero (utilizzato per esplorare dati), inoltre dato un albero è possibile definire una root (root) come il vertice da cui si parte per far partire il percorso, per cui ogni nodo può essere una radice e date due radici saranno presenti dei percorsi diversi.
L'albero implicitamente definisce un ordine dato dal nodo radice, per cui definito il nodo radice l'andamento è verso le foglie, quindi è possibile distinguere tra children (away from the root) e parents (towards the root), dove la radice è l'unico nodo senza parenti, ma presenta solo figli (vengono ordinati i nodi prima, è un ordinamento locale).
Un grafo pesato è un grafo in cui a ciascun ramo (arco) è associato un peso numerico, è quindi un tipo particolare di grafo etichettato in cui le etichette sono numeri (solitamente considerati positivi).

TdP 7.1 - libreria

La libreria NetworkX presenta tutta una serie di funzioni fatte a posta per definire e lavorare sui grafi, per cui la visione dei grafi presente su NetworkX viene detta nodo-centrica, per cui i nodi devono essere hashable e gli archi vengono visti come delle tuple tra nodi dello stesso tipo di oggetto.

Per prima cosa bisogna importare la libreria *import networkx as nx*, in seguito si può definire il grafo istanziando *nx.Grap()* che restituisce un grafo vuoto (senza nodi e edge), per cui è possibile andare ad istanziare direttamente gli archi come *g.add_edge(nodo1, nodo2, weight)*, in quanto se il nodo non esiste networkx lo istanzia e ci mette una connessione, inoltre per ogni edge è possibile definire un peso e sono presenti delle funzioni built in per definire i percorsi.
nota: https://networkx.org/en/ sito per documentazione di networkx

È possibile definire diverse tipologie di grafi come *nx.Graph()* (undirected), *nx.DiGraph()* (directed), *nx.MultiGrap()* (supports multiple edges between nodes), *nx.MultiDiGrap()* (directed multigraph).
È possibile definire gli edge e i nodi come *g.add_edge(n, n, weight)* e *g.add_node()* che permetto di aggiungere un nodo/edge per volta, oppure tramite *g.add_nodes_from(lista nodi), g.add_edges_from(lista di tuple)* e *g.add_weighted_edges_from([(nodo, nodo, weight), (...)])* che permettono di aggiungere una lista di nodi/edge.
nota: gli edge sono definiti come la coppia di nodi che sono stati inseriti

I grafi sono dei dizionari che contengono a loro volta dei dizionari, per cui le chiavi sono i nodi e i valori contengono un dizionario avente come chiave tutti i nodi connessi al nodo considerato e come valori i parametri degli edge (come il peso), quindi si può usare *g.nodes()* e *g.edge()* per avere informazioni sui nodi/edge, inoltre:

- g[n] yields a dictionary where keys are all the nodes connected with n (adjacency) and values are the edges params (like weight)
- g[u][v] yields the edge attributes
- n in g tests if node n is in g
- for n in g: iterates through the graph
- for nbr in g[n]: iterates through the neighbors of n

Classic graph operations:

- subgraph(G, nbunch) - induce subgraph of G on nodes in nbunch
- union(G1,G2) - graph union
- disjoint_union(G1,G2) - graph union assuming all nodes are different
- compose(G1,G2) - combine graphs identifying nodes common to both
- complement(G) - graph complement
- create_empty_copy(G) - return an empty copy of the same graph class
- convert_to_undirected(G) - return an undirected representation of G
- convert_to_directed(G) - return a directed representation of G

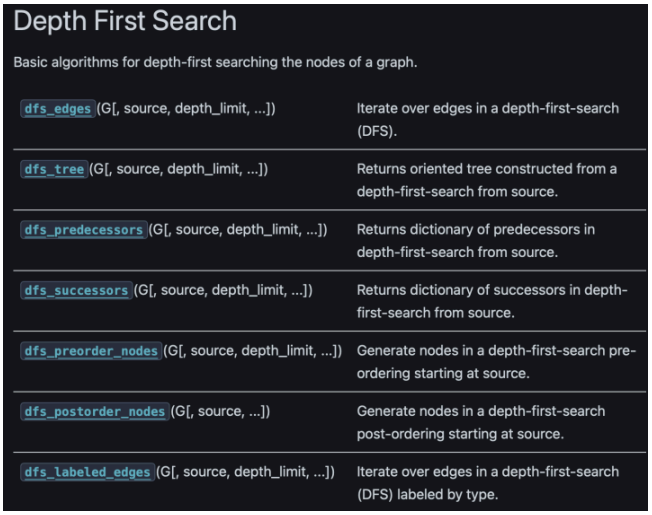
TdP 7.2 - visits

Visitare un grafo significa vuol dire esplorare in maniera sistematica il nodo andando ad analizzare le connessioni al nodo source tenendo in conto la direzione degli archi se presente, per cui è possibile adottare due strategie, Breadth-first visit (“in ampiezza”) e Depth-first visit (“in profondità”).
La visita in ampiezza funziona per livelli di esplorazione del grafico, per cui si parte da un nodo source andando ad osservare i nodi connessi al variare del numero di livelli, dove al primo livello sono visibili i soli nodi vicini, al secondo i vicini dei vicini del nodo source (raggiungibili con due passi), finché non si finiscono tutti i nuovi nodi che si sta esplorando, ovvero tutti i nodi connessi a quello che si sta esplorando, ottenendo come risultato i nodi ordinati in funzione della distanza dal nodo sorgente.
Il risultato di una BFV individua un "albero di visita" nel grafo, dove la radice dell'albero è il vertice sorgente, i nodi dell'albero sono tutti i vertici del grafo (appartenenti alla stessa componente connessa della sorgente), mentre gli archi dell'albero costituiscono un sottoinsieme degli archi del grafo, per cui sono un sottografo del grafo di partenza avente come nodi tutti quelli appartenenti alla componente connessa del nodo sorgente e gli archi sono il numero minimo di archi che permette di connettere il nodo source con qualsiasi altro nodo nel minor numero di passi.
La procedura BFS calcola tutti i cammini minimi per tutti i vertici in termini di numeri di archi, a partire dal vertice sorgente, dove con cammino minimo si indica il numero minimo di archi in un qualsiasi cammino tra due vertici

nota: in un grafo grafo non pesato lunghezza del cammino = numero di archi, mentre se gli archi sono pesati il cammino minimo è il cammino che minimizza il peso degli archi

Nella visita in profondità DFS ad ogni step si visita uno dei nodi vicini adiacente all'ultimo nodo analizzato, creando delle catene di nodi, finché non finisce il numero di nodi vicini all'ultimo nodo osservato, in seguito si torna indietro di un passo e si provano altre strade in maniera iterativa finchè non sono finiti tutti i nodi della componente connessa del nodo da cui si sta partendo, quindi finito l'algoritmo restituisce un albero di visita.
For all (w : adjacent_to(v)): If(not visited (w)) → Visit (w), DFS(w)

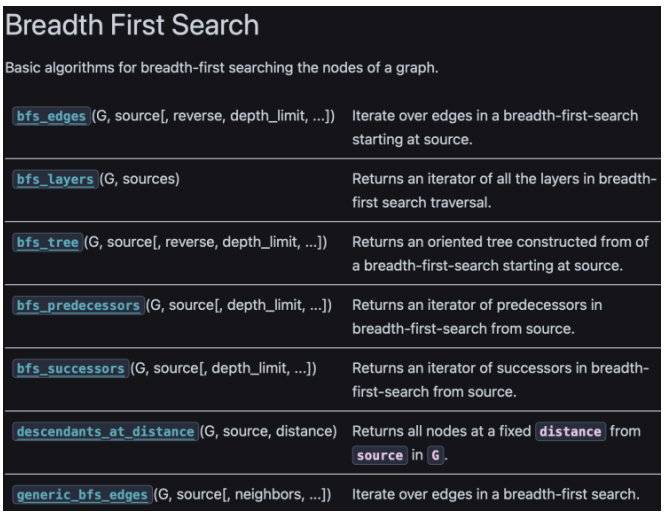
BFV utilizzato tipicamente per cercare i cammini minimi *tree = nx.bfs_tree(grafo, source)* (più lento), mentre DFS viene utilizzato quando si vuole analizzare il maggior numero di nodi nel minor tempo possibile *tree = nx.bfs_tree(grafo, source)* , tipicamente quando si vuole trovare una componente connessa. Nel caso dei grafi diretti bisogna considerare anche l'andamento delle frecce, per cui gli alberi a cui si converge dipende dal nodo sorgente considerato.



Depth First Search

Basic algorithms for depth-first searching the nodes of a graph.

`dfs_edges`(G[, source, depth_limit, ...])	Iterate over edges in a depth-first-search (DFS).
`dfs_tree`(G[, source, depth_limit, ...])	Returns oriented tree constructed from a depth-first-search from source.
`dfs_predecessors`(G[, source, depth_limit, ...])	Returns dictionary of predecessors in depth-first-search from source.
`dfs_successors`(G[, source, depth_limit, ...])	Returns dictionary of successors in depth-first-search from source.
`dfs_preorder_nodes`(G[, source, depth_limit, ...])	Generate nodes in a depth-first-search pre-ordering starting at source.
`dfs_postorder_nodes`(G[, source, ...])	Generate nodes in a depth-first-search post-ordering starting at source.
`dfs_labeled_edges`(G[, source, depth_limit, ...])	Iterate over edges in a depth-first-search (DFS) labeled by type.



Breadth First Search

Basic algorithms for breadth-first searching the nodes of a graph.

`bfs_edges`(G, source[, reverse, depth_limit, ...])	Iterate over edges in a breadth-first-search starting at source.
`bfs_layers`(G, sources)	Returns an iterator of all the layers in breadth-first search traversal.
`bfs_tree`(G, source[, reverse, depth_limit, ...])	Returns an oriented tree constructed from of a breadth-first-search starting at source.
`bfs_predecessors`(G, source[, depth_limit, ...])	Returns an iterator of predecessors in breadth-first-search from source.
`bfs_successors`(G, source[, depth_limit, ...])	Returns an iterator of successors in breadth-first-search from source.
`descendants_at_distance`(G, source, distance)	Returns all nodes at a fixed `distance` from `source` in `G`.
`generic_bfs_edges`(G, source[, neighbors, ...])	Iterate over edges in a breadth-first search.

TdP 7.3 - shortest path

I cammini ottimi sono cammini che ottimizzano una certa funzione obiettivo, per cui è necessario definire il peso di un cammino, andando a considerare un grafo diretto e pesato con una funzione costo W:E→IR secondo cui ogni arco ha associato un peso che è un valore reale, per cui il peso di un cammino p è la somma dei pesi degli archi lungo gli archi che compongono il cammino w(p) = Σw(u,v).
Il cammino minimo tra il vertice u e il vertice v è definito come il cammino di peso minimo tra u e v, qualora tale cammino esista, per cui il peso del cammino minimo è indicato con d(u,v), mentre se v non è raggiungibile da u, allora (per definizione) d(u,v)=∞.
Cercare il cammino minimo vuol dire cercare delle successioni di archi che minimizzano il peso del cammino, per cui esistono due varianti dell'algoritmo, distinti in Single-source shortest path (SS-SP), secondo cui dati u e v trova il cammino minimo tra u e v (i nodi di ricerca sono vincolati), oppure dato u trova il cammino minimo tra u e qualsiasi altro vertice, e All-pairs shortest path (AP-SP), secondo cui dato un grafo trova il cammino minimo tra qualsiasi coppia di vertici.

Per rappresentare i cammini minimi si usa come struttura dati, per cui dato il vertice sorgente u per ogni vertice v viene salvato il suo predecessore in una variable p(v) e il peso del cammino tra u e v in una variabile d(u,v), mentre per i multigrafi al posto di salvare il nodo precedente si salva l'arco.
nota: il vertice predecessore in un cammino minimo non dipende dalla destinazione finale.

Shortest path graph: Consider a source node u → Compute all shortest paths from u → Consider the relation Ep = { (v.preceding, v) } → Ep⊆E → Vp = { v∈V : v reachable from u } e Gp = G(Vp, Ep) is a subgraph of G(V,E) → Gp: the predecessor-subgraph (sottografo dei predecessori).

Gp è un albero (in virtù del Lemma) radicato in u, per cui in Gp i cammini (unici) che partono da u sono sempre cammini minimi, inoltre Gp non è unico, ma tutti i possibili Gp sono equivalenti (stesso peso per ogni cammino minimo), ovvero dipende dal nodo sorgente considerato.
Se G è un grafo non pesato, i cammini minimi possono essere calcolati anche mediante una visita in ampiezza.
Si considera un grafo orientato e pesato secondo una funzione peso sugli archi e si identifica il cammino minimo dal nodo v1 a vn, per cui dato un cammino minimo tra v1 e vk se prendo un sottoinsieme del cammino minimo esso risulta essere ancora ottimo, quindi se scegliamo arbitrariamente il vertice u', allora per tutti gli archi (u',v)∈E possiamo affermare che d(s,v)<=d(s,u')+w(u',v). Generalizzando, se scegliamo arbitrariamente un vertice u', per ogni arco (u',v) ∈ E vale la disuguaglianza d(s,v) ≤ d(s,u') + w(u',v).

La maggior parte degli algoritmi per il cammino minimo si basa sulla tecnica del rilassamento:

- Il vettore d[u] rappresenta d(s,u)
- Si mantiene una stima aggiornata d[u] del cammino minimo verso ciascun nodo u
- Si rilassa (ovvero si aggiorna) d[v] (e di conseguenza il predecessore p[v]) ogni volta che si scopre che il nodo v è raggiungibile più convenientemente percorrendo l'arco (u,v)

L'idea alla base del rilassamento è mantenere, per ciascun nodo u, una stima aggiornata d[u] del cammino minimo verso quel nodo (che rappresenta d(s,u)), insieme al relativo predecessore p[u]. Ogni volta che si scopre che un nodo v è raggiungibile in modo più conveniente passando per l'arco (u,v), si aggiorna (si "rilassa") il valore di d[v] e, di conseguenza, il predecessore p[v]. L'inizializzazione della struttura dati avviene ponendo d[v] = ∞ e p[v] = NIL per ogni vertice v tranne la sorgente s, per cui si pone d[s] = 0:

```
Initialize-Single-Source(G(V,E), s):
        for v in V:
        d[v] = ∞
        p[v] = NIL
        d[s] = 0
```

L'operazione di rilassamento di un arco (u,v) di peso w si definisce così:

```
Relax(u, v, w):
        if d[v] > d[u] + w(u,v):
        d[v] = d[u] + w(u,v)
        p[v] = u
```

Valgono inoltre alcuni lemmi generali sul rilassamento: dopo aver rilassato un arco (u,v) vale sempre d[v] ≤ d[u] + w(u,v); se il grafo non contiene cicli di peso negativo raggiungibili da s, il sottografo dei predecessori resta sempre un albero radicato in s, qualunque sequenza di rilassamenti venga applicata; infine, per ogni vertice v si ha sempre d[v] ≥ d(s,v), e se ad un certo punto d[v] diventa uguale al valore reale d(s,v), tale valore non cambierà più a seguito di ulteriori rilassamenti.
nota: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

I vari algoritmi per il calcolo dei cammini minimi si distinguono principalmente per l'ordine in cui applicano le operazioni di rilassamento, per il fatto di risolvere il problema single-source oppure all-pairs, e per la loro applicabilità o meno a grafi con pesi negativi (purché privi di cicli negativi).

Floyd-Warshall

L'algoritmo di Floyd-Warshall risolve il problema all-pairs shortest path (AP-SP): calcola cioè il cammino minimo tra ogni coppia di vertici del grafo. Si basa su una matrice dist[i][j] di dimensione n×n che, al termine dell'esecuzione, contiene la lunghezza del cammino minimo dal vertice vi al vertice vj; se dist[u][v] risulta ∞, significa che non esiste alcun cammino da u a v. Per poter ricostruire il cammino effettivo (e non solo il suo peso), viene mantenuta anche una matrice pred[s][j], che memorizza il predecessore del nodo vj lungo il cammino minimo che parte dalla sorgente vs.
L'algoritmo procede in due fasi: nella fase di inizializzazione la matrice dist viene riempita con i pesi degli archi esistenti (∞ altrove, 0 sulla



diagonale), mentre nella fase di rilassamento per ciascun vertice t del grafo (a turno) si verifica se passare attraverso t permette di accorciare il cammino tra ogni coppia (u,v).
L'algoritmo esegue in sostanza tre cicli annidati, ciascuno dei quali itera su tutti i vertici del grafo: la complessità risultante è quindi $O(V^3)$.
In NetworkX l'algoritmo è disponibile come nx.floyd_warshall(G, weight="weight"), che restituisce un dizionario, indicizzato per sorgente e destinazione, con le distanze dei cammini minimi.
Floyd-Warshall è particolarmente indicato per grafi densi o con pesi negativi, casi in cui Dijkstra non è applicabile — a patto naturalmente che non siano presenti cicli negativi.

Bellman-Ford-Moore

L'algoritmo di Bellman-Ford-Moore risolve invece il problema single-source shortest path (SS-SP) ed è anch'esso basato sulla tecnica del rilassamento: per ogni vertice, rilassa tutti gli archi possibili. A differenza di Dijkstra, funziona correttamente anche in presenza di archi di peso negativo, ed è in grado di rilevare la presenza di cicli di peso negativo (nel qual caso il problema del cammino minimo non è ben definito), pur non essendo comunque in grado di risolverlo. La sua complessità è $O(V \cdot E)$.
In NetworkX è disponibile, la funzione nx.all_pairs_bellman_ford_path(G, weight="weight"), che calcola i cammini minimi tra tutte le coppie di nodi restituendo un iteratore di coppie (sorgente, dizionario dei cammini verso ogni destinazione).

```python
def Bellman_Ford_Moore(V, E, s, w):

    #Initialization
    dist[s] = 0             #set distance to source = 0
    for v in V-{s}:         #set all other dist to infty and predecessors
to None
        dist[v] = ∞
        pred[v] = None

    #Relax edges repeatedly
    for i in range(1, len(V)):
        for (u, v) in E:
            if dist[v] > dist[u] + w(u, v):
                d[v] = d[u] + w(u, v) #set new shortest path value
                pred[v] = u #update the predecessor

    for (u, v) in E:
        if dist[v] > dist[u] + w(u, V):
            PANIC!

    return dist, pred
```

Algoritmo di Dijkstra
L'algoritmo di Dijkstra risolve il problema single-source shortest path (SS-SP) e funziona sia su grafi diretti sia su grafi non diretti. La condizione fondamentale per la sua applicabilità è che tutti gli archi abbiano peso non negativo: in presenza di pesi negativi l'algoritmo fallirebbe, producendo risultati scorretti. Ad ogni passo sceglie il nodo non ancora visitato con distanza stimata minima e garantisce comunque di trovare la soluzione ottima.
Nella sua implementazione più semplice l'algoritmo ha complessità $O(E + V^2)$, poiché ad ogni passo occorre scandire l'intero insieme dei nodi non ancora visitati per trovare quello con distanza minima; utilizzando invece una coda con priorità (min-heap) è possibile ottenere un'implementazione più efficiente, con complessità $O(E + V \cdot \log V)$.
In NetworkX, la funzione nx.dijkstra_path(G, source, target, weight="weight") restituisce la lista di nodi che compongono il cammino minimo pesato da source a target, mentre nx.dijkstra_path_length(G, source, target, weight="weight") ne restituisce solo il peso complessivo.

```python
def Dijkstra(V, E, s, w):

    #Initialise
    dist[s] = 0
    Q = []
    for v in V-{s}:
        dist[v] = ∞       # set initial dist to infty
        prev[v] = None    # set predecessors to None
        Q.append(v)       # Build a list of unvisited nodes

    #Run relaxation iteration
    while Q is not empty:
        u = q in Q with min dist[q]     # Pick one element of Q
        Q.remove(u)

        for v in neighborhood(u) still in Q:    # Cycle on neighbors of k
            newDist = dist[u] + w(u,v)          # Verify if path through u-v is better
            if newDist < dist[v]:
                dist[v] = newDist               # Update the new shortest path
                prev[v] = k

    return dist, pred
```

La tabella seguente riassume gli algoritmi visti, il tipo di problema che risolvono (single-source o all-pairs), la loro complessità computazionale e le eventuali limitazioni sul tipo di grafo su cui possono essere applicati:
Per la loro implementazione pratica, NetworkX mette a disposizione un intero modulo dedicato ai cammini minimi (networkx.algorithms.shortest_paths), che include tutte le varianti descritte sopra oltre a numerose funzioni di utilità.

Algorithm	Problem	Efficiency	Limitation
Floyd-Warshall	AP	$O(V^3)$	No negative cycles
Bellman-Ford	SS	$O(V \cdot E)$	No negative cycles
Repeated Bellman-Ford	AP	$O(V^2 \cdot E)$	No negative cycles
Dijkstra	SS	$O(E + V \cdot \log V)$	No negative edges
Repeated Dijkstra	AP	$O(V \cdot E + V^2 \cdot \log V)$	No negative edges
Breadth-First visit	SS	$O(V + E)$	Unweighted graph

Cicli nei grafi

Un ciclo di un grafo (o circuito) è un sottoinsieme dell'insieme degli archi che forma un cammino il cui primo nodo coincide con l'ultimo. A seconda delle proprietà richieste si distinguono diverse categorie di cicli e cammini particolari.

- Ciclo hamiltoniano: un ciclo che utilizza ogni vertice del grafo esattamente una volta.
- Cammino hamiltoniano: analogo al ciclo hamiltoniano, ma senza il vincolo di tornare al punto di partenza: è un cammino tra due vertici che visita ogni vertice del grafo esattamente una volta.
- Cammino euleriano: una passeggiata sugli archi del grafo che utilizza ogni arco del grafo originale esattamente una volta.
- Ciclo euleriano (o circuito euleriano): un cammino euleriano che parte e termina nello stesso vertice. Un grafo che ammette un ciclo euleriano si dice grafo euleriano.

Un grafo semplice e connesso è euleriano se e solo se non possiede vertici di grado dispari, cioè se tutti i vertici hanno grado pari. Questo risultato è oggi formalizzato in un teorema più generale: un grafo connesso ammette un ciclo euleriano se e solo se tutti i suoi vertici hanno grado pari, mentre ammette un cammino euleriano (non necessariamente chiuso) se e solo se possiede al più due vertici di grado dispari.

Algoritmo di Hierholzer per i cicli euleriani

Per calcolare un ciclo euleriano in un grafo euleriano G si utilizza tipicamente l'algoritmo di Hierholzer.
Si parte scegliendo un vertice qualsiasi v e seguendo un cammino di archi non ancora utilizzati fino a tornare a v: poiché tutti i vertici hanno grado pari, non è possibile restare bloccati in un vertice diverso da v, dato che ogni volta che il cammino entra in un nodo w deve esistere ancora un arco inutilizzato che ne esce.
Il ciclo così ottenuto è chiuso, ma potrebbe non coprire ancora tutti i vertici e gli archi del grafo di partenza, per cui finchè esiste un vertice v che appartiene al ciclo corrente ma possiede ancora archi adiacenti non inclusi nel ciclo, si parte un nuovo cammino da v seguendo archi non utilizzati fino a tornare a v, e si unisce il ciclo così ottenuto a quello precedente. In pseudocodice:

1. Si identifica un circuito in G, chiamato R1. Si marcano gli archi di R1 come visitati; sia i=1.
2. Se Ri contiene tutti gli archi di G, l'algoritmo termina.
3. Altrimenti, sia vi un nodo di Ri incidente ad un arco ei non ancora marcato.
4. Si costruisce un nuovo circuito Qi, a partire da vi, usando l'arco ei. Si marcano gli archi di Qi come visitati.
5. Ri+1 si ottiene congiungendo, nel nodo vi, i circuiti Ri e Qi.
6. Si incrementa i e si torna al passo 2.

NetworkX mette a disposizione un intero sottomodulo dedicato ai cicli euleriani, con funzioni come is_eulerian(G) (verifica se G è euleriano), eulerian_circuit(G) (restituisce un iteratore sugli archi di un ciclo euleriano), eulerize(G) (trasforma un grafo qualunque in un grafo euleriano), is_semieulerian(G), has_eulerian_path(G) ed eulerian_path(G).

Cicli hamiltoniani e il problema del commesso viaggiatore

Per i cicli hamiltoniani la situazione è ben diversa: esistono sì dei teoremi che permettono di stabilire se un grafo è hamiltoniano (cioè se contiene almeno un ciclo hamiltoniano), ma non esiste, nel caso generale, alcuna soluzione efficiente per trovare effettivamente un tale ciclo.
L'approccio classico per la risoluzione esatta del TSP (Traveling Salesman Problem) è il backtracking: si mantiene traccia del miglior tour trovato fino a quel momento, e la ricerca prosegue esplorando nuove soluzioni parziali solo quando il loro costo, per quanto già accumulato, è ancora inferiore al costo del miglior tour trovato; in caso contrario si effettua un backtrack, abbandonando il ramo di ricerca corrente.
NetworkX ne implementa diversi, tra cui: l'algoritmo di Christofides, che fornisce una 3/2-approssimazione della soluzione ottima su grafi completi (funzione christofides(G, weight, tree)); un approccio goloso (greedy_tsp); tecniche metaeuristiche come il simulated annealing (simulated_annealing_tsp) e il threshold accepting (threshold_accepting_tsp); e infine l'algoritmo di Asadpour per il caso asimmetrico (asadpour_atsp).
La funzione generica traveling_salesman_problem(G, weight, ...) permette inoltre di applicare questi metodi anche a grafi incompleti, convertendo preventivamente il problema in uno equivalente su un grafo completo tramite il calcolo dei cammini minimi tra tutte le coppie di nodi, e poi riportando la soluzione trovata sul grafo originale.